

Act-0 Data Pipelines and Architectures

A teaching guide for the team: NMT, TUAB and stroke data, EEG-VJEPA, FEI+C and the evaluation heads

EEE 402 (G1) — Artificial Intelligence and Machine Learning Laboratory
Group 01 · Bangladesh University of Engineering and Technology (BUET)

Internal team document. The TUAB numbers come from an unlicensed Kaggle copy and must not be shared outside the group.

Contents: 1. NMT · 2. TUAB · 3. Stroke · 4. V-JEPA · 5. FEI+C · 6. Evaluation heads

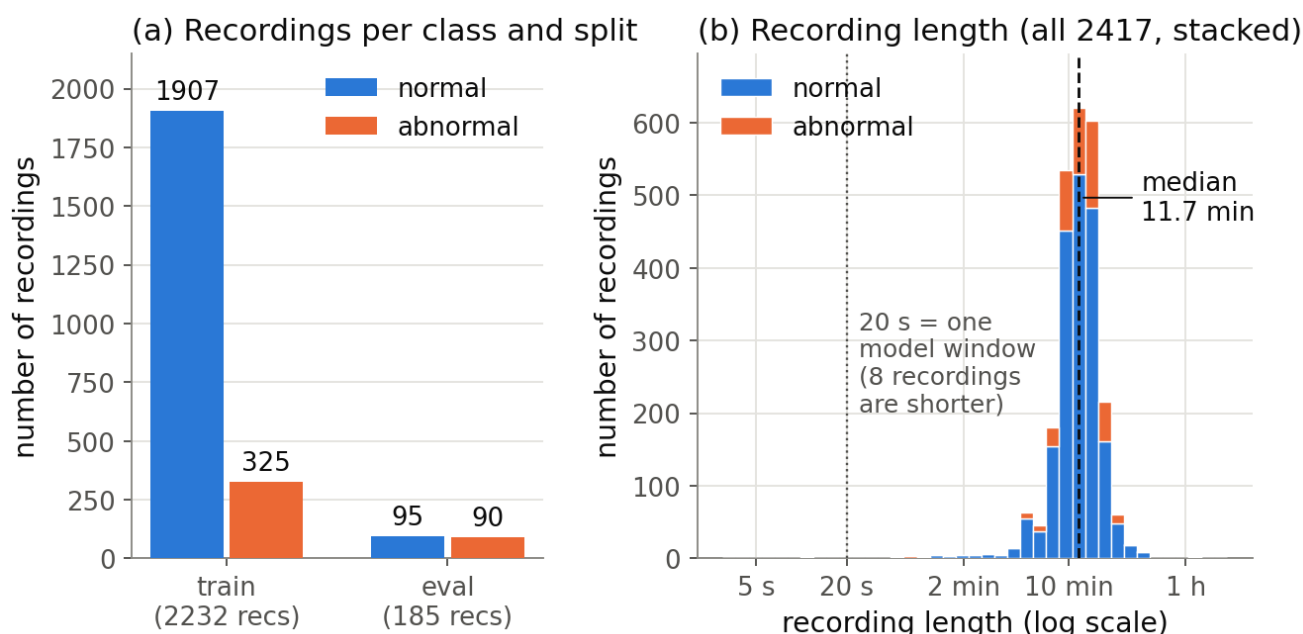
The grey code boxes are copied word for word from the repository when this PDF is built. Each box names its file (under `code/`) and line numbers, so you can open the same lines in the editor. `...` on its own line marks skipped lines. Files under `src/` and `app/` are the EEG-VJEPA authors' code; the rest is ours.

NMT Data Pipeline

The **NMT** dataset (NMT Scalp EEG, Pakistan) is a large collection of clinical EEG recordings, each labelled *normal* or *abnormal*. In Act-0 it is used to **pre-train our own FEI and Branch C encoders** (see "FEI+C Architecture"). EEG-VJEPA was not pre-trained on NMT by us.

Dataset Overview

- **Total Recordings:** 2,417 recordings
- **Split:** train has 1,907 normal + 325 abnormal recordings; eval has 95 normal + 90 abnormal.
- **Length:** about 12 minutes per recording (median 11.7 min); ~489 hours in total.
- **Sampling Rate:** 200 Hz
- **Channels:** The raw files contain 21 channels (e.g., FP1|FP2|F3|...|A1|A2). We drop the two ear references (A1, A2) and keep the **19 standard 10-20 scalp channels**.



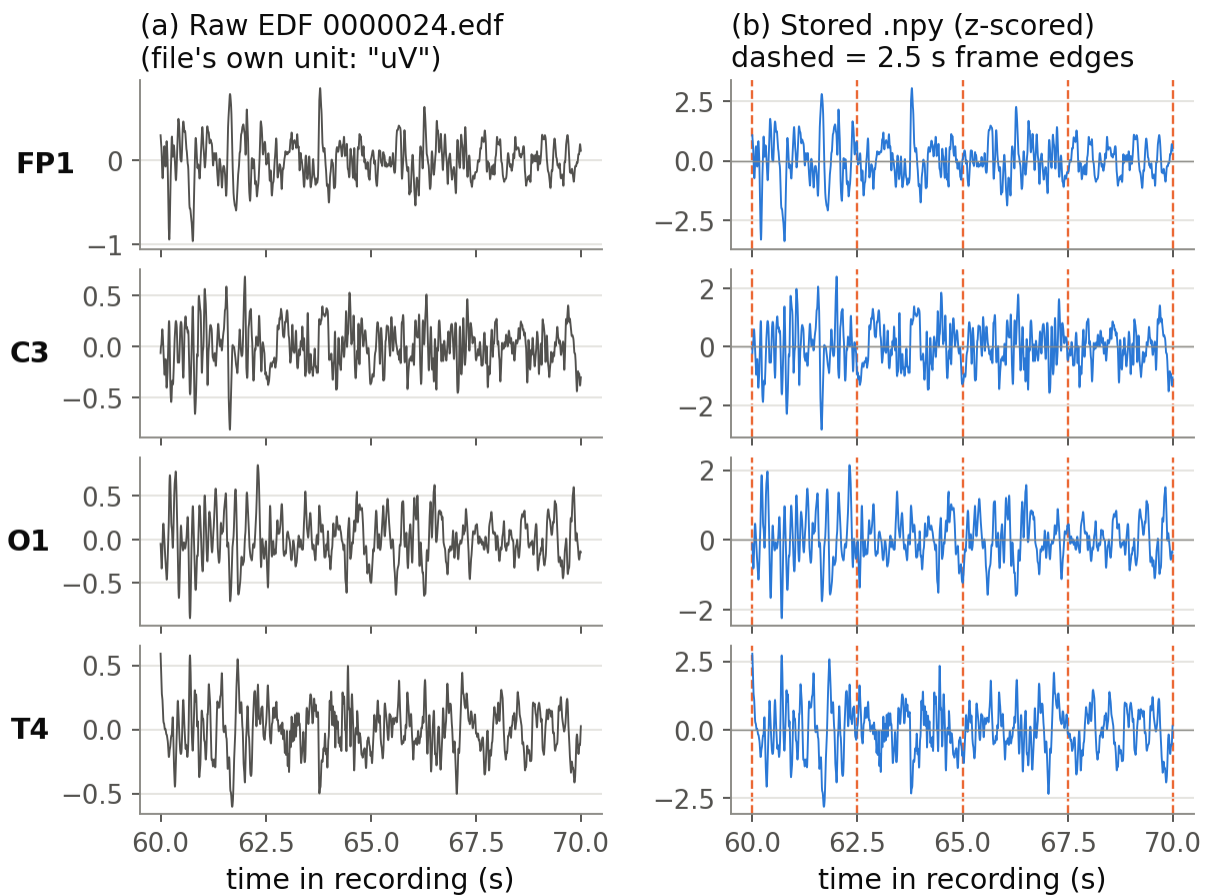
Preprocessing

NMT preprocessing is deliberately light. We use the dataset's own NMT_preprocessed files, where each channel is only **z-scored** (mean 0, spread 1). **No band-pass filter** is applied, so noise and artifacts are still present. The z-score only puts all channels and recordings on the same amplitude scale.

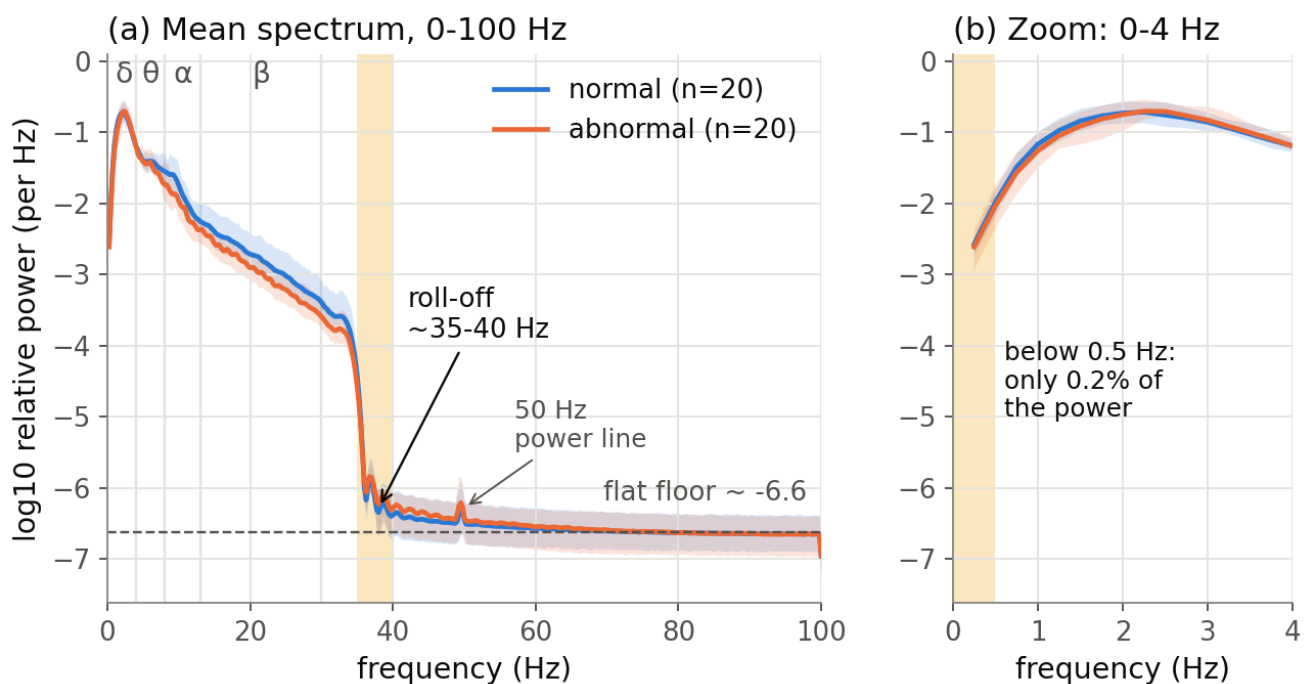
In code, DATA points at these files. `load_continuous` joins a recording's stored (frames, 19, 500) array back into one continuous (19, samples) signal:

code/fei_pretrain.py, lines 36–37, 50–54

```
DATA = "/home/mashfiq/eeg_vjepa/data/NMT_preprocessed"
FS, NCH = 200, 19
...
def load_continuous(path):
    """(F,19,500) z-scored frames -> full continuous (19, F*500) float32.
    Used only by the probe (one recording at a time -> bounded memory)."""
    x = np.load(path) # (F, 19, 500)
    return x.transpose(1, 0, 2).reshape(x.shape[1], -1).astype(np.float32)
```



The **PSD** (power spectral density) plot shows how much power the signal has at each frequency. Because no filter is used, the NMT spectrum keeps its full shape. This spectrum matters later: FEI and Branch C learned their statistics from it, so any new dataset fed to them must have a similar spectrum (see "Stroke", PSD matching).



Windowing and Folds

Instead of feeding a whole 12-minute recording into the model at once, we cut it into **20 s windows** (19 channels $\times$ 4,000 samples):

- **Pre-training:** random 20 s windows are drawn from the training recordings.
- **Embedding a recording:** it is cut into non-overlapping 20 s windows; each window is encoded and the embeddings are averaged into one vector.

Pre-training reads one random window per item. `load_window` memory-maps the file and reads only the 2–3 stored frames that the window spans:

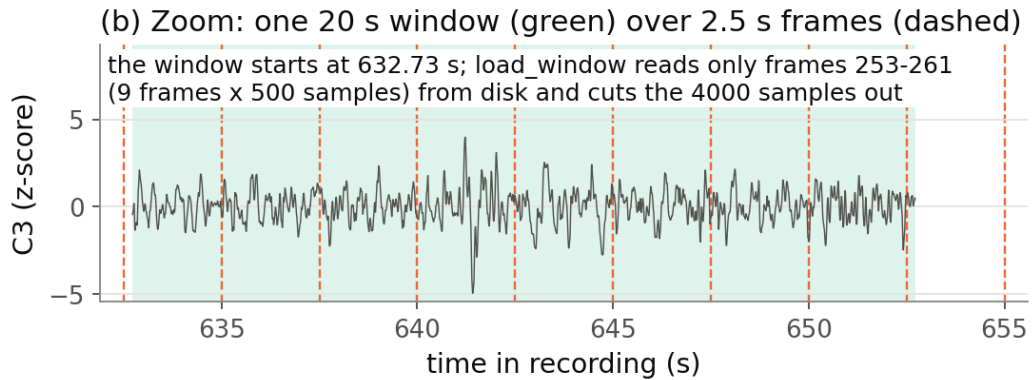
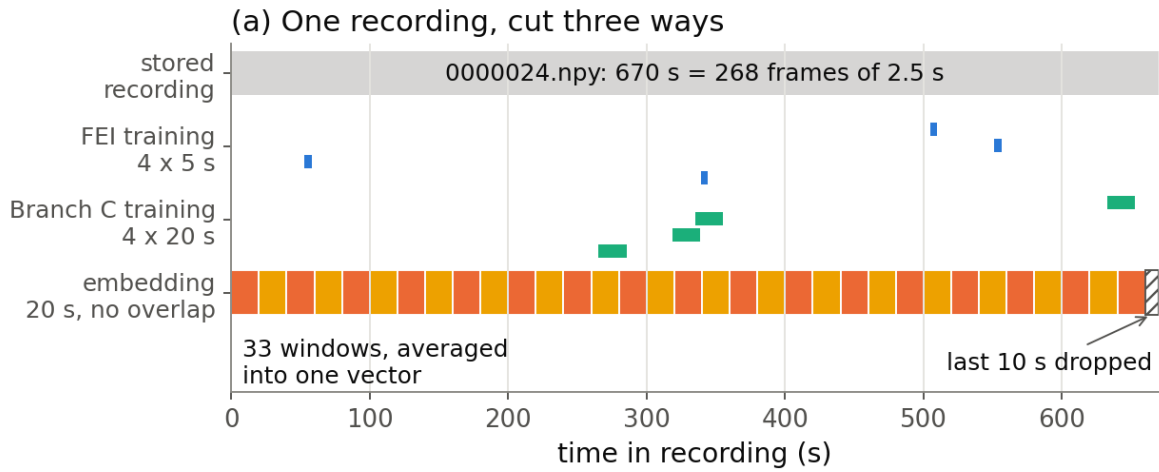
code/fei_pretrain.py, lines 57–71

```
def load_window(path, L):
    """Random length-L continuous window, reading ONLY the 2-3 frames it spans
    via mmap (~114KB), not the whole recording. This is the speedup + the OOM fix:
    per-item memory is bounded regardless of recording length."""
    x = np.load(path, mmap_mode="r") # (F,19,500), no full read
    Fr, C, W = x.shape
    T = Fr * W
    if T < L: # short -> wrap-pad (rare)
        cont = np.asarray(x).transpose(1, 0, 2).reshape(C, -1)
        return np.ascontiguousarray(np.tile(cont, (1, L // T + 1))[:, :L]).astype(np.float32)
    s = random.randint(0, T - L)
    f0, f1 = s // W, (s + L - 1) // W # frames covering [s, s+L)
    chunk = np.asarray(x[f0:f1 + 1]).transpose(1, 0, 2).reshape(C, -1) # (19, nf*500)
    off = s - f0 * W
    return np.ascontiguousarray(chunk[:, off:off + L]).astype(np.float32)
```

Embedding a recording cuts it into non-overlapping windows and averages the encoder outputs:

code/fei_pretrain.py, lines 179–189

```
def embed_recording(enc, path, L):
    """Mean-pooled encoder embedding over non-overlapping L-windows."""
    sig = load_continuous(path)
    T = sig.shape[1]
    if T < L:
        sig = np.tile(sig, (1, L // T + 1))[:, :L]; T = L
    starts = range(0, T - L + 1, L)
    w = np.stack([sig[:, s:s + L] for s in starts]) # (W,19,L)
    with torch.no_grad():
        e = enc(torch.from_numpy(w).to(DEV)).mean(0) # (d,)
    return e.cpu().numpy()
```



To test on unseen recordings, we use **5-fold cross-validation (CV)**. The train and eval recordings are pooled (2,417) and split into 5 folds with StratifiedKFold (seed 0), so each fold keeps the same normal/abnormal ratio. In fold k , the fold- k recordings (about 484) are the test set and the other $\sim 1,933$ are for training. For fold k we pre-trained a separate encoder (f0–f4) that **never saw the fold- k recordings**. These five fold encoders are the ones later ensembled on TUAB and stroke.

The fold loop in `run_cv.pretrain` receives only the training folds' files, and each fold's encoder is saved as `..._cv_s0_f{k}.pt`:

code/fei_pretrain.py, lines 270–279

```
files = list_split("train") + list_split("eval")    # pooled, same order as band-power
y = np.array([c for _, c in files])
skf = StratifiedKFold(n_splits=5, shuffle=True, random_state=args.cv_seed)
for k, (tr, te) in enumerate(skf.split(np.zeros(len(y)), y)):
    if str(k) in state:
        print(f"[f{k+1}] done (AUC={state[str(k)]['auc']:.3f}) -> skip", flush=True); continue
    tr_files = [files[i] for i in tr]; te_files = [files[i] for i in te]
    print(f"\n==== FOLD {k+1}/5 (train {len(tr)} / test {len(te)}) =====", flush=True)
    enc = pretrain(tr_files, args, tag=f"[f{k+1}] "); enc.eval()
    torch.save(enc.state_dict(), args.ckpt.replace(".pt", f"_cv_s{args.cv_seed}_f{k}.pt"))
```

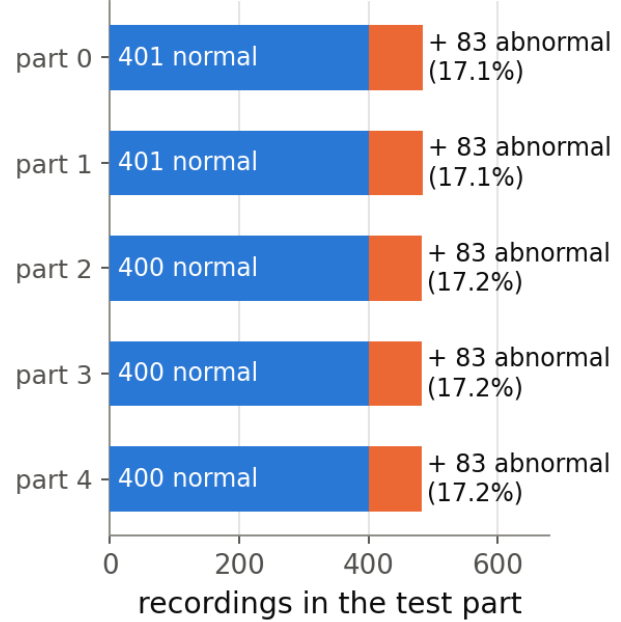
(a) First 40 recordings in the list and the part (0-4) each was put in, at random

1 2 3 0 1 1 1 4 2 0 3 1 3 3 3 0 1 1 2 2 2 3 2 0 0 3 0 4 3 1 2 3 3 1 1 0 4 1 2 2

(b) Encoder f_k never sees part k

fold encoder	f0	test	train	train	train	train
	f1	train	test	train	train	train
	f2	train	train	test	train	train
	f3	train	train	train	test	train
	f4	train	train	train	train	test
		part 0	part 1	part 2	part 3	part 4

(c) What is inside each test part



TUAB Data Pipeline

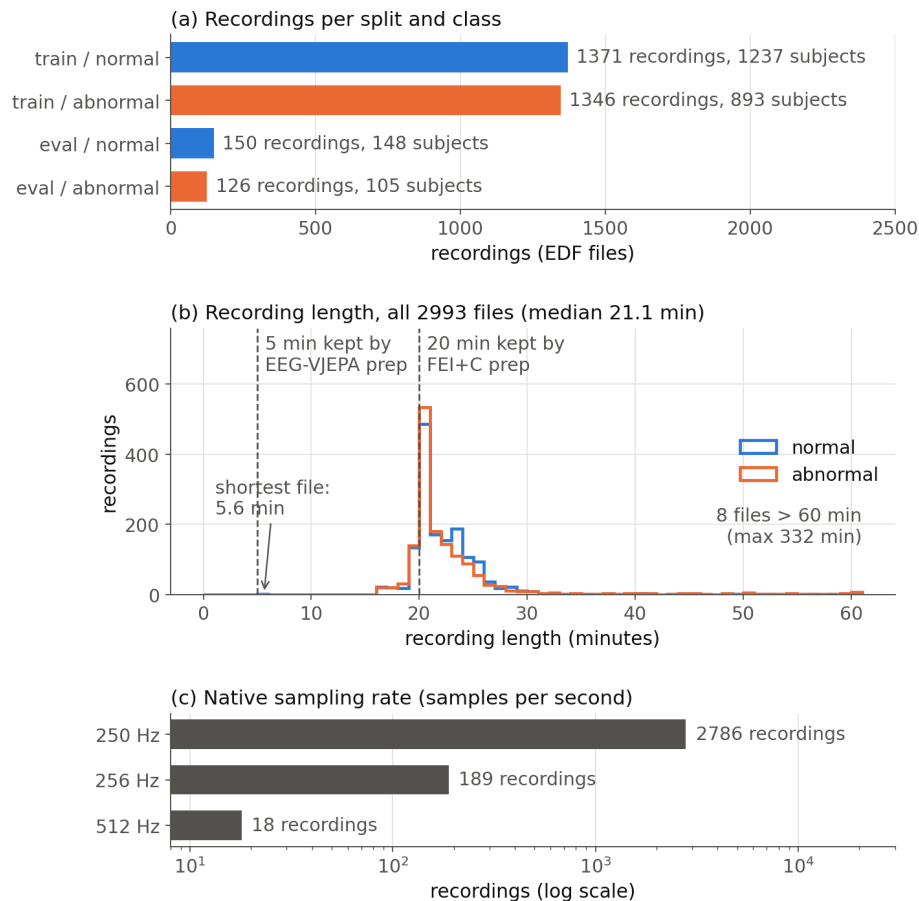
The **TUAB** dataset (TUH Abnormal EEG Corpus v3.0.1, Temple University Hospital, USA) is the standard benchmark for normal vs abnormal EEG. It is the dataset of **Part 1**: we test whether the released EEG-VJEPa checkpoints reach the paper's TUAB results.

- **Labels:** each recording is labelled *normal* or *abnormal* from the neurologist's clinical report.
- **Official split:** train has 1,371 normal + 1,346 abnormal recordings; eval has 150 normal + 126 abnormal. There are 2,329 patients, and no patient appears in both train and eval, so the test is on unseen people.
- **Sampling rates:** most files are 250 Hz (2,786). Some are 256 Hz (189) or 512 Hz (18).
- **Channels:** files have about 30 channels named like EEG_FP1-REF. We keep the 19 standard 10-20 scalp channels.

Our TUAB copy comes from Kaggle and is not officially licensed. It is for internal team use only; do not share TUAB numbers outside the group.

Dataset and Preprocessing

Different sampling rates, extra channels and different amplitudes would confuse a network. So every recording is converted to exactly the format the released EEG-VJEPa checkpoints were pre-trained on (paper §4.1).



Our pipeline performs the following steps:

1. **Channel selection:** keep the 19 scalp channels, in a fixed order. Every recording then has the same channels in the same positions.
2. **Cropping:** keep the **first 300 s (5 min)**, so every recording has the same length.
3. **Band-pass filter 1–40 Hz:** remove slow drift below 1 Hz, and muscle noise and the 60 Hz power-line hum above 40 Hz. The brain rhythms (delta to beta) lie inside this band.
4. **Resample to 100 Hz:** 100 samples per second are enough for signals up to 40 Hz, and fewer samples mean less computation.
5. **Z-score each channel:** subtract the channel's mean and divide by its standard deviation. Every channel then has mean 0 and spread 1, which makes channels and recordings comparable.
6. **Framing:** cut the recording into **5 s frames** (500 samples) that start every 2.5 s (50 % overlap). This gives an array of shape (frames, 19, 500).

All six steps are one function, `preprocess_edf` in `preprocess_nmt.py`; TUAB calls the same function. The settings:

code/preprocess_nmt.py, lines 29–38

```
CHANNELS = [
    'FP1', 'FP2', 'F3', 'F4', 'C3', 'C4', 'P3', 'P4',
    'O1', 'O2', 'F7', 'F8', 'T3', 'T4', 'T5', 'T6',
    'FZ', 'CZ', 'PZ'
]
WINDOW_SIZE = 500      # 5 seconds at 100 Hz (matches paper checkpoints)
# Paper preprocessing (arXiv:2507.03633 §4.1), reproduced exactly:
CROP_SECONDS = 300     # crop to a fixed 5-minute length
BANDPASS = (1.0, 40.0) # bandpass 1-40 Hz (encoder was pretrained on this band)
WINDOW_STRIDE = 250    # 50% overlap -> ~118 windows per 5 min (paper's 118x19x500)
```

The function body, steps 1–6 in order:

code/preprocess_nmt.py, lines 43–79

```
raw = mne.io.read_raw_edf(edf_path, preload=False, verbose=False)
if ch_map: # e.g. TUAB 'EEG FP1-REF' -> 'FP1'
    raw.rename_channels({k: v for k, v in ch_map.items() if k in raw.ch_names})

# Pick channels (ignore missing if any, but they should be there)
raw.pick_channels(CHANNELS)

# Reorder to match CHANNELS exactly
raw.reorder_channels(CHANNELS)

# Paper §4.1, in order: crop to fixed 5 min -> bandpass 1-40 Hz ->
# downsample to 100 Hz -> channel-wise z-score -> overlapping 5 s windows.
t0 = min(crop_start, max(0.0, raw.times[-1] - CROP_SECONDS)) # rec too short for the offset: keep its
last 5 min
raw.crop(tmin=t0, tmax=min(t0 + CROP_SECONDS, raw.times[-1]))
raw.load_data(verbose=False)
raw.filter(BANDPASS[0], BANDPASS[1], verbose=False)
if round(raw.info['sfreq']) != TARGET_FS:
    raw.resample(TARGET_FS)

# Get data: shape (n_channels, n_times)
data = raw.get_data()

# Per-channel z-score normalization
mean = np.mean(data, axis=1, keepdims=True)
std = np.std(data, axis=1, keepdims=True)
std[std == 0] = 1.0
data = (data - mean) / std

# Overlapping 5 s windows -> (num_frames, 19, 500)
n_channels, n_times = data.shape
if n_times < WINDOW_SIZE:
    return False # under one window after cropping -> skip
num_frames = (n_times - WINDOW_SIZE) // WINDOW_STRIDE + 1
frames = np.stack([
    data[:, i * WINDOW_STRIDE : i * WINDOW_STRIDE + WINDOW_SIZE]
    for i in range(num_frames)
]).astype(np.float32) # (num_frames, 19, 500)
```

TUAB channel names look like EEG FP1-REF. preprocess_tuab.py passes this map so that they become FP1:

code/preprocess_tuab.py, line 22

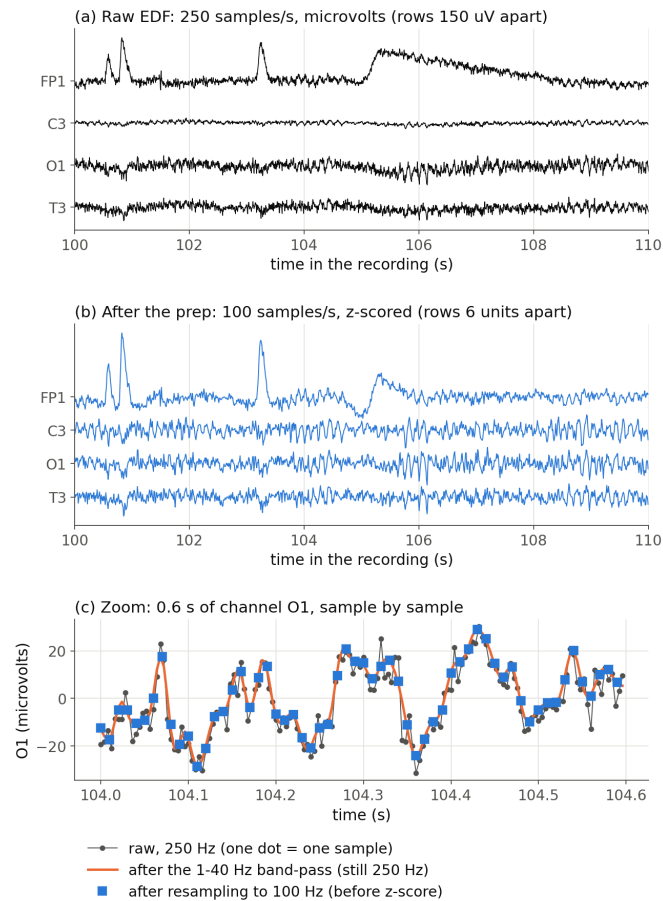
```
CH_MAP = {f"EEG {c}-REF": c for c in P.CHANNELS} # 01_tcp_ar naming
```

(FEI+C uses a *separate* TUAB prep matched to NMT: first 20 min, 0.5–40 Hz, 200 Hz, z-score, 2.5 s frames. It is not used by EEG-VJEPA.)

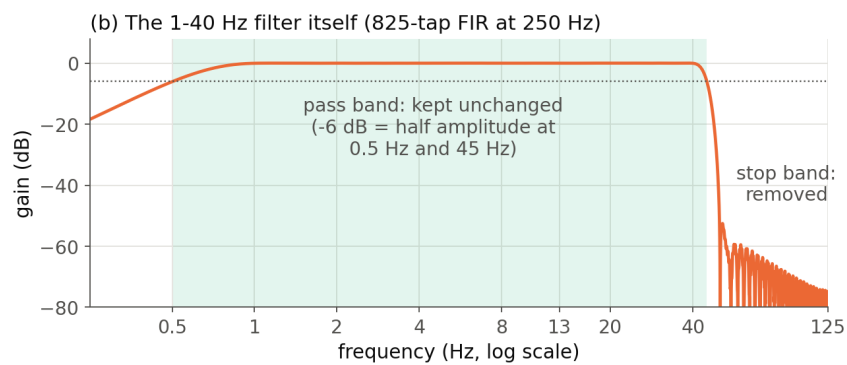
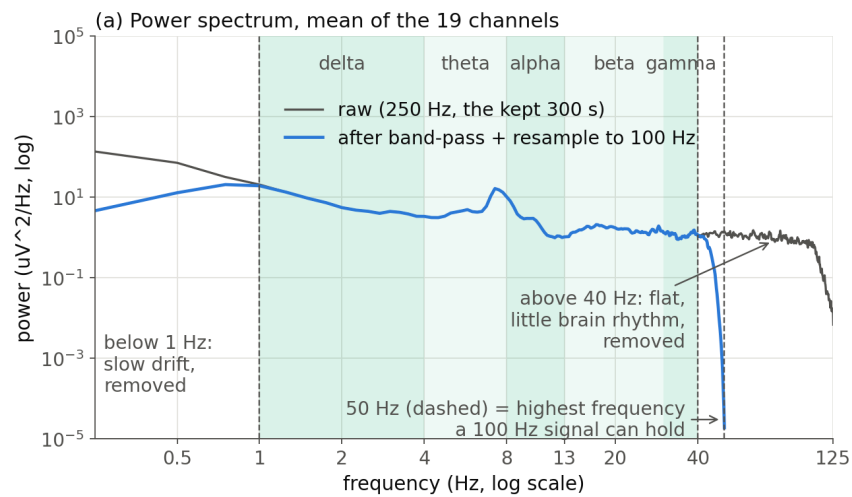
That separate prep, from tuab_feic.py (non-overlapping 500-sample frames):

code/tuab_feic.py, lines 57–63

```
raw = mne.io.read_raw_edf(f, preload=False, verbose=False)
raw.rename_channels({k: v for k, v in CH_MAP.items() if k in raw.ch_names})
raw.pick(P.CHANNELS).reorder_channels(P.CHANNELS)
raw.crop(tmax=min(CROP_S, raw.times[-1])).load_data(verbose=False)
d = raw.filter(0.5, 40.0, verbose=False).resample(200, verbose=False).get_data()
d = (d - d.mean(1, keepdims=True)) / d.std(1, keepdims=True)
fr = d[:, :d.shape[1] // 500 * 500].reshape(19, -1, 500).transpose(1, 0, 2).astype(np.float32)
```

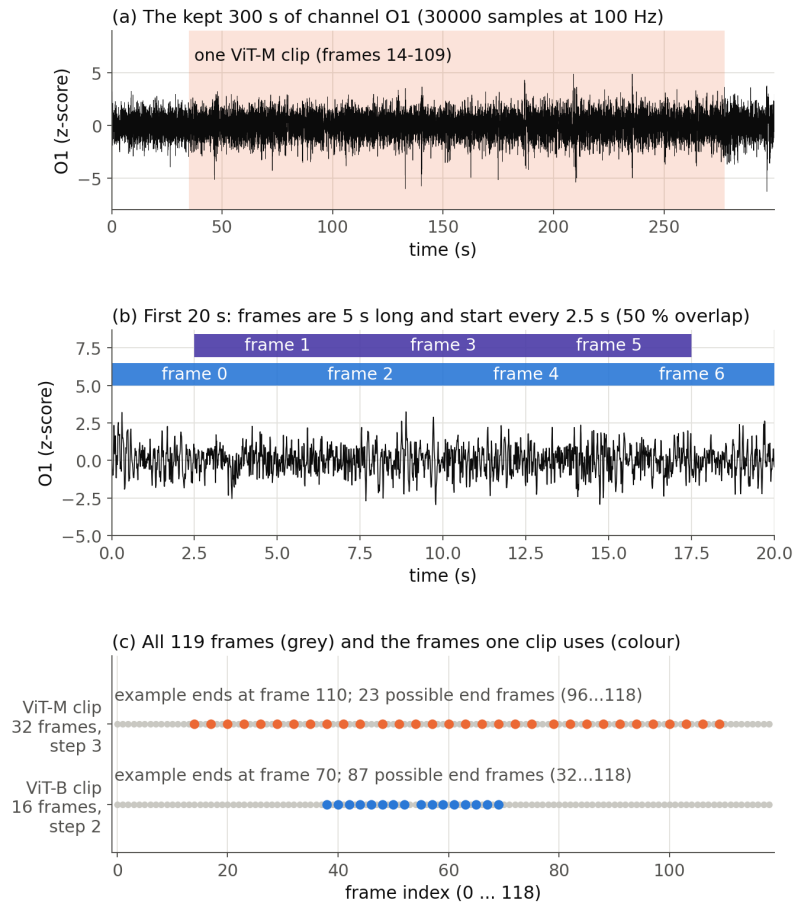


The **PSD** (power spectral density) shows how much power the signal has at each frequency. After preprocessing, the power outside 1–40 Hz should be strongly reduced, while the brain rhythms inside the band stay unchanged. The PSD figure lets us check that the filter did exactly this.



Framing and Clipping

After preprocessing, the 5-minute EEG segment is divided into smaller overlapping **frames**.



The network does not see single frames. It sees **clips**, short "videos" made of several frames:

- **ViT-M**: a clip is **32 frames**, taken every 3rd frame (frame_step = 3).
- **ViT-B**: a clip is 16 frames, taken every 2nd frame.

The input tensor is (batch, 1, 32, 19, 500) for ViT-M. During training and evaluation, a random clip position is drawn from each recording, and the head's output for that clip is the recording's prediction.

The clip settings per model, the loader (the authors' VideoDataset, one random clip per recording) and the index rule (clip): 32 indices spread evenly over the $32 \times 3 = 96$ frames before end:

code/tuab_cached_eval.py, lines 39–42

```
MODELS = {"vitm": dict(ckpt=f"{ROOT}/pretrained/eeg_vjepa_ViT-M_4x30x4.pth.tar", model_name="vit_small",
    frames_per_clip=32, frame_step=3, tubelet_size=4),
    "vitb": dict(ckpt=f"{ROOT}/pretrained/eeg_vjepa_ViT-B_4x30x2.pth.tar", model_name="vit_base",
    frames_per_clip=16, frame_step=2, tubelet_size=2)}
```

code/tuab_cached_eval.py, lines 60–64

```
def dataset(subset, split, M):
    # the upstream dataset -> identical sample order + labels (abnormal=0, normal=1)
    return vd.VideoDataset(data_paths=[f"{SUBS}/s{subset}"], frames_per_clip=M["frames_per_clip"],
        frame_step=M["frame_step"], num_clips=1, random_clip_sampling=True,
        allow_clip_overlap=True, transform=None, mode=split)
```

code/tuab_cached_eval.py, lines 67–71

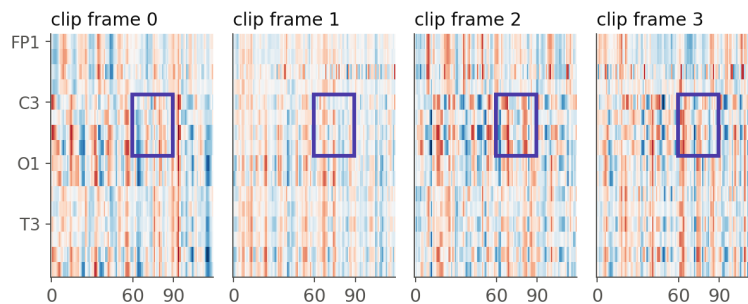
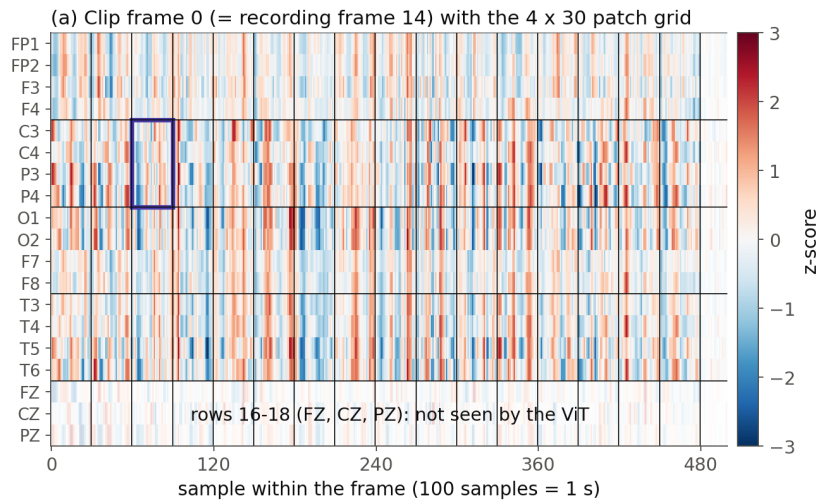
```
def clip(frames, end, M): # = VideoDataset.loadvideo_decord for partition len > clip_len
    fpc, clip_len = M["frames_per_clip"], M["frames_per_clip"] * M["frame_step"]
    start = end - clip_len
    idx = np.clip(np.linspace(start, end, num=fpc), start, end - 1).astype(np.int64)
    return frames[idx]
```

Training subsets: following the paper, we do not train the head on all 2,717 training recordings. Instead we draw **5 subsets (s0–s4)** of 546 recordings each (276 normal + 270 abnormal). Each subset is random but class-balanced, and has no patient overlap. Every result is then tested on the same 276 official eval recordings, and reported as mean $\pm$ sd over the 5 subsets.

How the subsets are built (`build_subsets` in `tuab_released_eval.py`): shuffle with seed `s`, take at most one recording per patient until the quotas (`N_TRAIN`: 276 normal, 270 abnormal) are met, and link the shared eval set:

code/tuab_released_eval.py, lines 35–59

```
def build_subsets():
    recs = [(lab, f) for lab in N_TRAIN for f in sorted(glob.glob(f"{PREP}/train/{lab}/*.npz"))]
    for s in SEEDS:
        rng = random.Random(s)
        order = recs[:]
        rng.shuffle(order)
        used, take = set(), {lab: [] for lab in N_TRAIN}
        for lab, f in order:  # <=1 recording per subject, class quotas as in the paper
            subj = os.path.basename(f).split("_")[0]
            if subj in used or len(take[lab]) >= N_TRAIN[lab]:
                continue
            used.add(subj); take[lab].append(f)
        d = f"{SUBS}/{s}/{s}"
        for lab, fs in take.items():
            assert len(fs) == N_TRAIN[lab], (s, lab, len(fs))
            os.makedirs(f"{d}/train/{lab}", exist_ok=True)
            for f in fs:
                link = f"{d}/train/{lab}/{os.path.basename(f)}"
                if not os.path.lexists(link):
                    os.symlink(f, link)
            if not os.path.lexists(f"{d}/eval"):
                os.symlink(f"{PREP}/eval", f"{d}/eval")
        n = {lab: len(os.listdir(f"{d}/train/{lab}")) for lab in N_TRAIN}
        assert n == N_TRAIN, n
        print(f"s{s}: {n} + eval -> {PREP}/eval")
```



(b) Clip frames 0-3, first 16 channels, samples 0-119. The purple box, taken in all 4 frames (4 frames x 4 channels x 30 samples = 480 numbers), becomes ONE token (tubelet size 4).

Stroke Data Pipeline

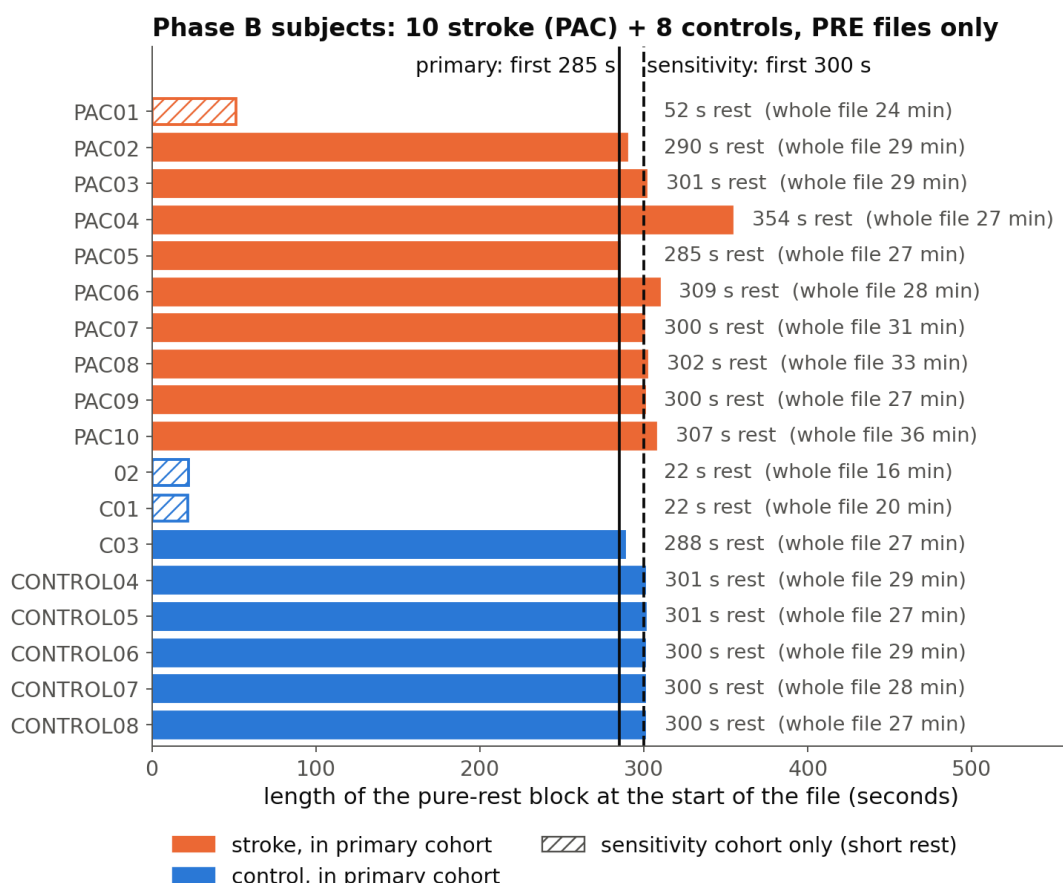
The stroke evaluation (**Part 2 / Phase B**) asks one question: can frozen EEG features tell **stroke patients** from **healthy controls**?

- **Data:** Zenodo record 19599466, resting-state EEG in BrainVision format.
- **Recording:** 63 channels, 1,000 Hz, referenced to the right ear (A2).
- **Sessions:** only the **PRE** (before therapy) files are used, so every subject contributes one recording.

Cohorts

- **Primary cohort:** 9 patients (PAC02–PAC10) vs 6 controls (C03, CONTROL04–08). Every subject gives the same fixed **285 s** of rest, so no class has more data than the other.
- **Sensitivity cohort:** 10 patients vs 8 controls, using the first 300 s. It checks that the result does not depend on which subjects were included.

With so few subjects, every result is a small-sample result.



Preprocessing and PSD Matching

Channels that the dataset authors marked as bad are first rebuilt from their neighbours (**spherical-spline interpolation**). After that, the same recording is prepared in **three different ways**, one for each model:

load (in stroke_phaseB.py) reads the BrainVision file, crops the rest period, marks the authors' bad channels and interpolates them:

code/stroke_phaseB.py, lines 59–69

```
def load(subj, secs):
    import mne
    raw = mne.io.read_raw_brainvision(f"{RAW}/{subj}.vhdr", preload=False, verbose=False)
    raw.crop(tmax=secs - 1 / raw.info["sfreq"]).load_data(verbose=False)
    raw.set_channel_types({c: "eeg" for c in raw.ch_names if "EOG" in c} | {"A1": "misc"})
    raw.set_montage("standard_1005", match_case=False, on_missing="ignore", verbose=False)
    bads = [b if isinstance(b, str) else raw.ch_names[b - 1] for b in BADS.get(bad_key(subj), [])]
    raw.info["bads"] = [b for b in bads if raw.get_channel_types([b])[0] == "eeg"]
    if raw.info["bads"]:
        raw.interpolate_bads(reset_bads=True, verbose=False)
    return raw, bads
```

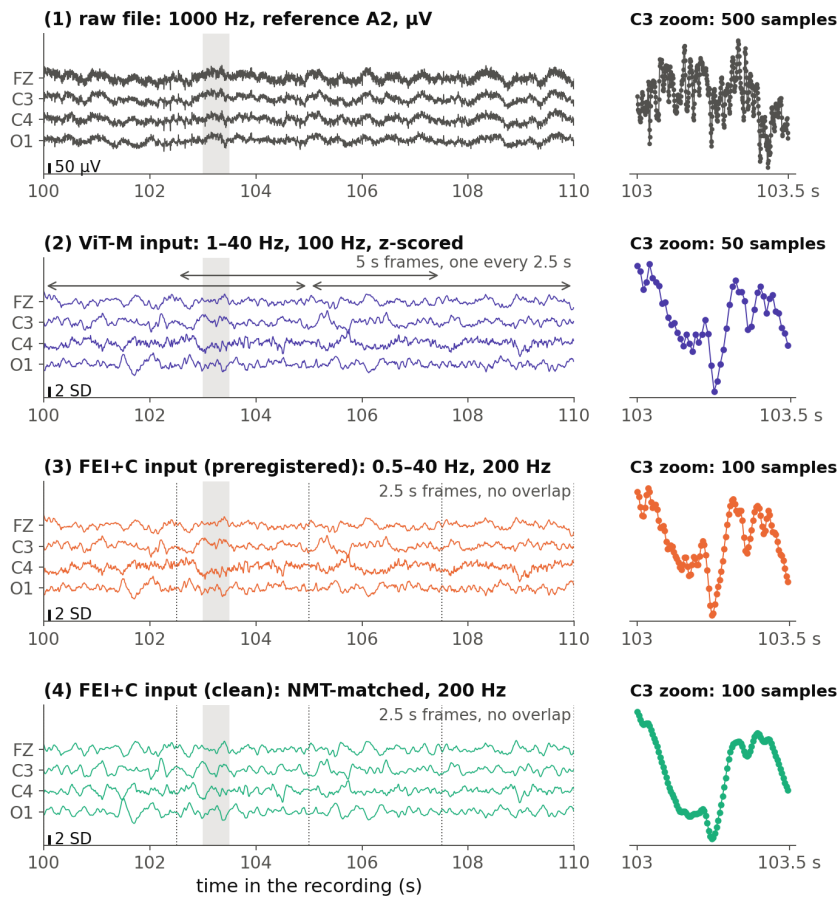
Prep	Used by	Steps
ViT-M prep	EEG-VJEPA	average reference, 19 channels (T7/T8/P7/P8 renamed T3/T4/T5/T6), 1–40 Hz, 100 Hz, z-score, 5 s frames every 2.5 s (same as TUAB)
FEI+C prep (pre-registered)	FEI+C	0.5–40 Hz, 200 Hz, 19 channels, z-score
Clean FEI+C prep (post-hoc)	FEI+C only	average reference, 19 channels, 200 Hz, no band-pass, PSD matching to NMT (below), z-score, 2.5 s frames

The BSI features (below) use their own prep on the original 63 channels.

The two pre-registered encoder preps come from one function. The loop runs over the vitm and feic settings (low cut-off, sampling rate, frame stride):

code/stroke_phaseB.py, lines 97–108

```
def encoder_preps(raw):
    """CAR over EEG ch -> 19 ch (CHANNELS order). vitm = $4.1 (1-40 Hz, 100 Hz, z, 5 s stride 2.5 s);
    feic = NMT-matched (0.5-40 Hz = NMT device band, 200 Hz, z, non-overlap 500)."""
    from preprocess_nmt import CHANNELS
    r = raw.copy().pick("eeg").set_eeg_reference("average", verbose=False).rename_channels(RENAME)
    r.pick(CHANNELS).reorder_channels(CHANNELS)
    out = {}
    for name, (lo, fs, stride) in {"vitm": (1.0, 100, 250), "feic": (0.5, 200, 500)}.items():
        d = r.copy().filter(lo, 40.0, verbose=False).resample(fs, verbose=False).get_data()
        d = (d - d.mean(1, keepdims=True)) / d.std(1, keepdims=True)
        out[name] = np.stack([d[:, i:i + 500] for i in range(0, d.shape[1] - 500 + 1,
            stride)]).astype(np.float32)
    return out
```



PSD matching is used **only in the clean FEI+C prep**. It was added after the pre-registered run, for this reason: after the pre-registered prep, the stroke spectrum was still far from NMT's. Branch C's BatchNorm then received inputs far outside the range it had learned on NMT, and Branch C collapsed (almost the same output for every subject). The clean prep builds **one fixed zero-phase filter per cohort** (per channel). It maps the cohort's average log-spectrum (all subjects pooled, labels never used) onto NMT's average log-spectrum. Zero-phase means the filter changes power at each frequency but does not shift the waveform in time. Because this step was decided after seeing results, clean-prep numbers are reported as post-hoc.

The clean prep is in `stroke_phaseB_clean.py`. `logpsd` is the Welch log-spectrum; `shape` multiplies each channel's FFT by a real, positive gain, which changes power but not timing (zero-phase):

`code/stroke_phaseB_clean.py`, lines 37–39

```
def logpsd(sig): # (19,T) -> (f, (19,F) log10 Welch PSD, 0.25 Hz bins)
    f, p = welch(sig, fs=FS, nperseg=NPER, axis=-1)
    return f, np.log10(p + 1e-20)
```

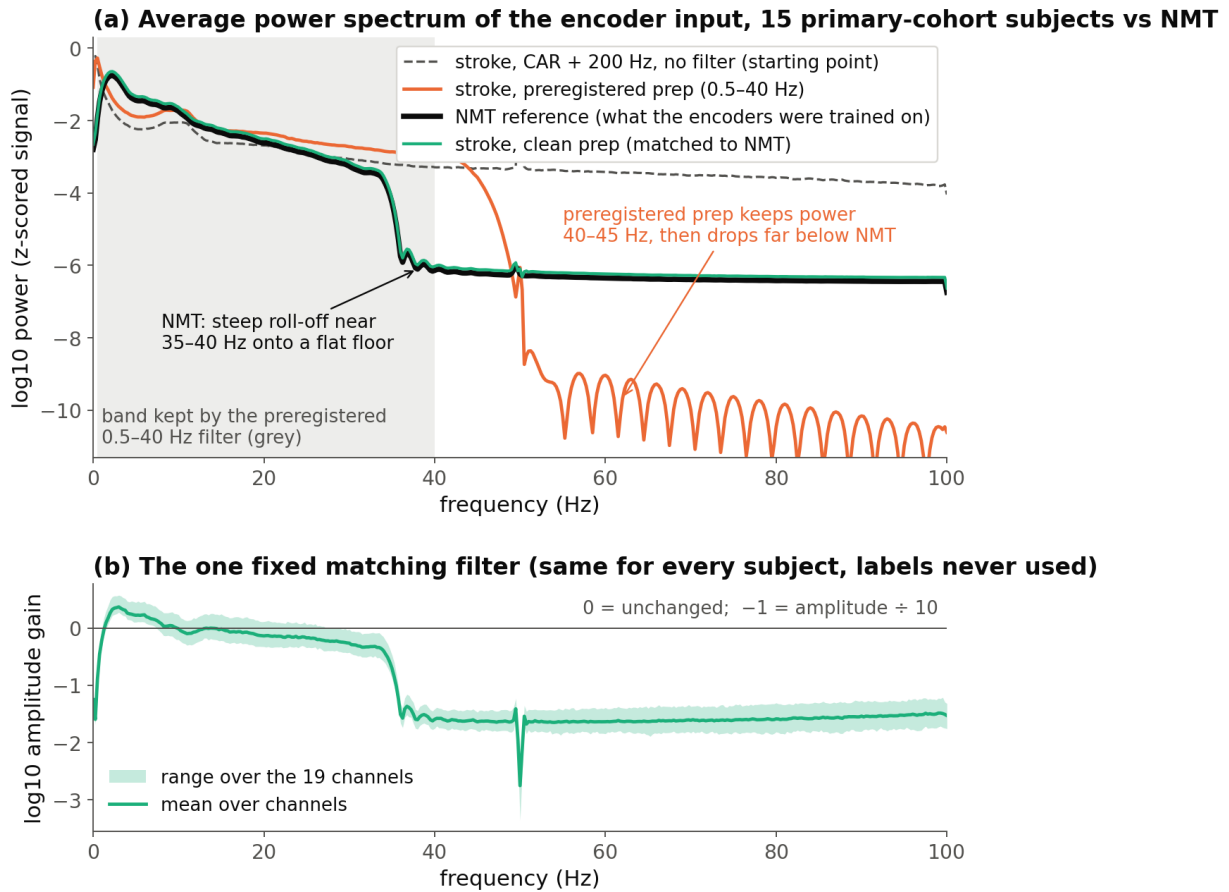
`code/stroke_phaseB_clean.py`, lines 42–47

```
def shape(sig, f, logh): # zero-phase per-channel spectral shaping; logh = log10 AMPLITUDE gain on
    pad = NPER
    x = np.pad(sig, ((0, 0), (pad, pad)), mode="reflect")
    g = np.fft.rfftfreq(x.shape[1], 1 / FS)
    H = 10 ** np.stack([np.interp(g, f, h) for h in logh])
    return np.fft.irfft(np.fft.rfft(x) * H, x.shape[1])[:, pad:-pad]
```

The gain logh is found by iteration ($N_ITER = 4$). Each round moves the cohort's mean log-spectrum half-way towards NMT's (ref). All subjects are pooled and the labels are never read:

code/stroke_phaseB_clean.py, lines 95–99

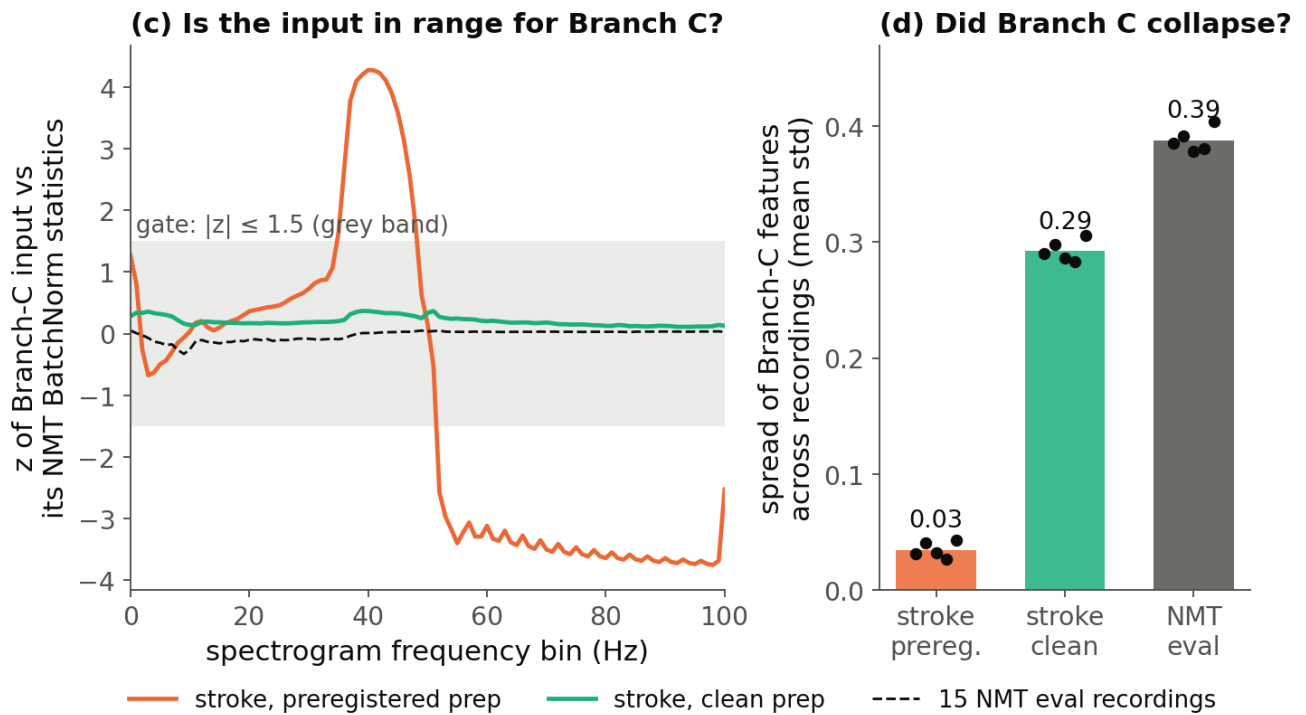
```
logh, lp0 = np.zeros_like(ref), np.stack([logpsd(x)[1] for x in S.values()])
for it in range(N_ITER): # iterate: re-z after shaping shifts the level slightly
    lp = np.stack([logpsd(zs(shape(x, f, logh)))[1] for x in S.values()])
    logh += (ref - lp.mean(0)) / 2
D = {s: zs(shape(x, f, logh)) for s, x in S.items()}
```



The **gates** are checks run before any classifier is trained. They confirm that the clean prep worked:

- the largest gap between the stroke and NMT log-spectra fell from 5.81 to 0.18 decades (gate: ≤ 0.25);
- Branch C's BatchNorm inputs fell from $|z| \approx 4.3$ to ≈ 0.37 ;
- no embedding dimension is constant across subjects (no collapse).

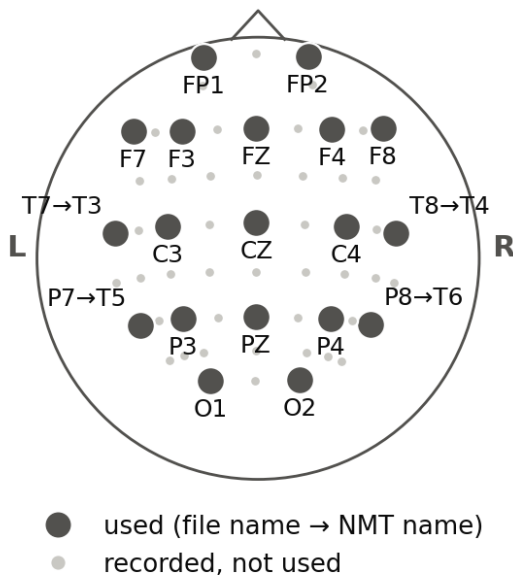
PSD matching does **not** remove artifacts; it only changes the overall spectral shape.



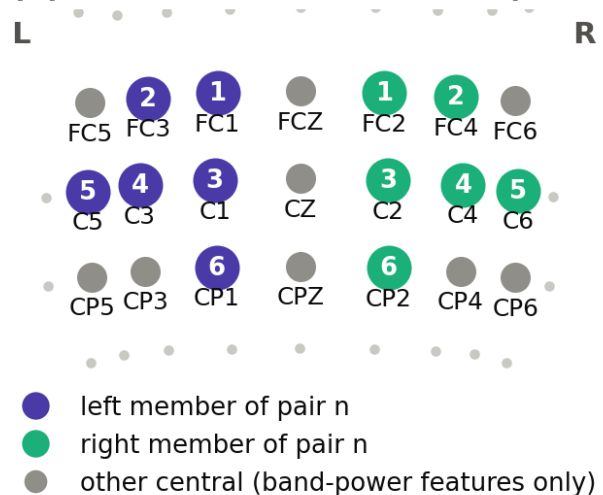
Montage and BSI

The recordings have **63 electrodes**. The encoders use a **19-channel** 10-20 subset, the same channels as NMT and TUAB. The BSI uses **21 central channels** (FC, C and CP rows) and forms **6 left/right pairs**: FC1/FC2, FC3/FC4, C1/C2, C3/C4, C5/C6 and CP1/CP2.

(a) 19 channels fed to the encoders



(b) BSI: 21 central channels (zoomed)



The **Brain Symmetry Index (BSI)** is a hand-crafted clinical feature. A stroke usually damages one side of the brain, so the two hemispheres' power becomes unequal. The BSI prep is: A2 reference, 50 Hz notch, 0.5–100 Hz band-pass, 2 s epochs, rejection of epochs beyond $\pm 100 \mu V$, then Welch power spectra. For each pair and frequency, with R and L the right and left power:

$$\text{pdBSI} = \text{mean of } |(R - L) / (R + L)|$$

It is 0 for perfect symmetry and grows towards 1 with asymmetry. We compute it over 1–30 Hz and in the delta, theta, alpha and beta bands.

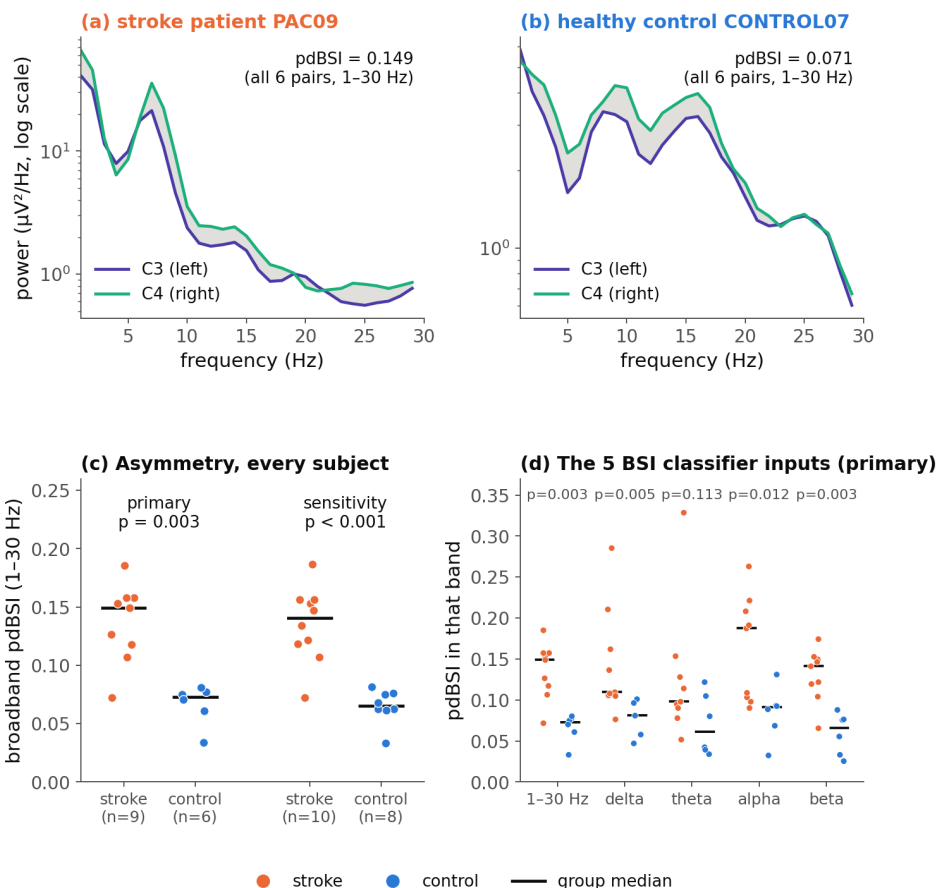
In code, `pdbsi` is the formula. `gate_feats` does the BSI prep and calls it for 1–30 Hz and each band; `AH` holds the left and `UH` the right channel of each pair:

code/stroke_phaseB.py, lines 93–94

```
def pdbsi(L, R):
    # pairwise-derived BSI (Sheorajpanday 2009): mean over pairs & bins of |(R-L)/(R+L)|, in [0,1]
    return np.abs((R - L) / (R + L)).mean()
```

code/stroke_phaseB.py, lines 74–79, 87–89

```
r = raw.copy().notch_filter(50, verbose=False).filter(0.5, 100, verbose=False)
fs = int(r.info["sfreq"])
x = r.get_data(picks=CENTRAL21) * 1e6 # (21, T) uV
ep = x[:, :x.shape[1] // (2 * fs) * 2 * fs].reshape(21, -1, 2 * fs).transpose(1, 0, 2)
keep = np.abs(ep).max((1, 2)) <= 100
f, p = welch(ep[keep], fs=fs, nperseg=fs, axis=-1) # 1 Hz bins
...
for name, (lo, hi) in [{"1_30", (1, 30)}] + list(BANDS.items()):
    sel = (f >= lo) & (f < hi)
    out[f"bsi_{name}"] = float(pdbsi(np.stack([ch[c][sel] for c in AH]), np.stack([ch[c][sel] for c in
UH])))
```



How the stroke numbers are produced

1. **One vector per subject:** encoder embeddings (averaged over the subject's windows, 5 fold encoders ensembled), BSI features, or both joined.
2. **Classifier:** StandardScaler + logistic regression, the same simple head as everywhere else.
3. **Leave-one-subject-out (LOSO):** train on all subjects but one, predict the left-out one, repeat for every subject, then compute one AUROC.

4. **Permutation test (1,000 shuffles)**: the labels are shuffled and the whole LOSO is rerun. The p-value is how often shuffled labels score as well as the real ones.
5. **Bootstrap CI**: subjects are resampled to give a confidence interval for the AUROC.
6. **Random-init control**: the same pipeline with untrained encoders.

loso is step 3. arm runs it for every ensemble member, averages their probabilities, and does steps 4–5. The `perm_p` line counts the shuffles that score at least as well as the real labels:

code/stroke_phaseB.py, lines 189–196

```
def loso(X, y):
    """Leave-one-subject-out out-of-fold P(stroke). One recording per subject -> L00."""
    p = np.zeros(len(y))
    for i in range(len(y)):
        tr = np.arange(len(y)) != i
        clf = make_pipeline(StandardScaler(), LogisticRegression(C=1.0, class_weight="balanced",
max_iter=5000))
        p[i] = clf.fit(X[tr], y[tr]).predict_proba(X[i:i + 1])[0, 1]
    return p
```

code/stroke_phaseB.py, lines 201–211

```
P = np.array([loso(X, y) for X in members]); p = P.mean(0)
null = []
for _ in range(N_PERM):
    cool_gate(pause=88.0, resume=78.0, abort=92.0, verbose=False)
    yp = rng.permutation(y)
    null.append(metrics(yp, np.mean([loso(X, yp) for X in members], 0))["auroc"])
m = metrics(y, p)
au = [metrics(y, q)["auroc"] for q in P]
return p, P, np.array(null), m | dict(auroc_ci=bootstrap_ci(y, p, "auroc", N_BOOT),
bal_acc_ci=bootstrap_ci(y, p, "bal_acc", N_BOOT),
perm_p=float((1 + np.sum(np.array(null) >= m["auroc"]))) / (1 + N_PERM)),
member_auroc=au, member_auroc_mean=float(np.mean(au)),
member_auroc_sd=float(np.std(au)))
```

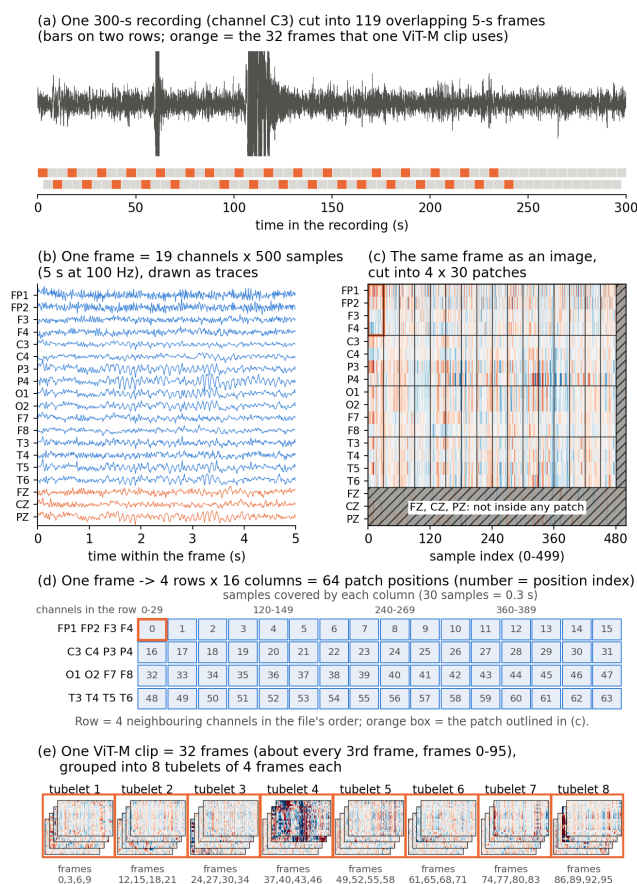
V-JEPA Architecture

V-JEPA (Video Joint-Embedding Predictive Architecture) is a self-supervised model first designed for video. **EEG-VJEPA** is the same model applied to EEG. In Act-0 we use the checkpoints **released by the paper's authors** (mainly ViT-M); we did not pre-train it ourselves.

EEG as Video

To use a video model, the EEG is treated as a video:

- **One frame** = one 5 s piece of EEG, a 19×500 "image" (19 channels $\times$ 500 samples at 100 Hz).
- **One clip** = 32 frames in a row (ViT-M), like 32 video frames.



The model can then look at patterns across channels (space) and across frames (time) at the same time.

Network Blocks

The clip is first cut into small pieces called **tokens**. One Conv3d layer does this:

- a patch covers **4 channels $\times$ 30 samples** of a frame, and a **tubelet** joins 4 consecutive frames;
- this gives 8 (time) $\times$ 4 (channel groups) $\times$ 16 (sample groups) = **512 tokens**, each described by **384 numbers**;
- the patches do not overlap and there is no padding, so the last 3 channels and the last 20 samples of each frame are not covered.

The tokeniser in the authors' code is one Conv3d whose kernel equals its stride, so the patches cannot overlap. Our loader builds ViT-M with `patch_size=(4, 30)` and `tubelet_size=4`:

code/src/models/utils/patch_embed.py, lines 56–66

```
self.proj = nn.Conv3d(
    in_channels=in_chans,
    out_channels=embed_dim,
    kernel_size=(tubelet_size, patch_size[0], patch_size[1]),
    stride=(tubelet_size, patch_size[0], patch_size[1]),
)

def forward(self, x, **kwargs):
    B, C, T, H, W = x.shape
    x = self.proj(x).flatten(2).transpose(1, 2)
    return x
```

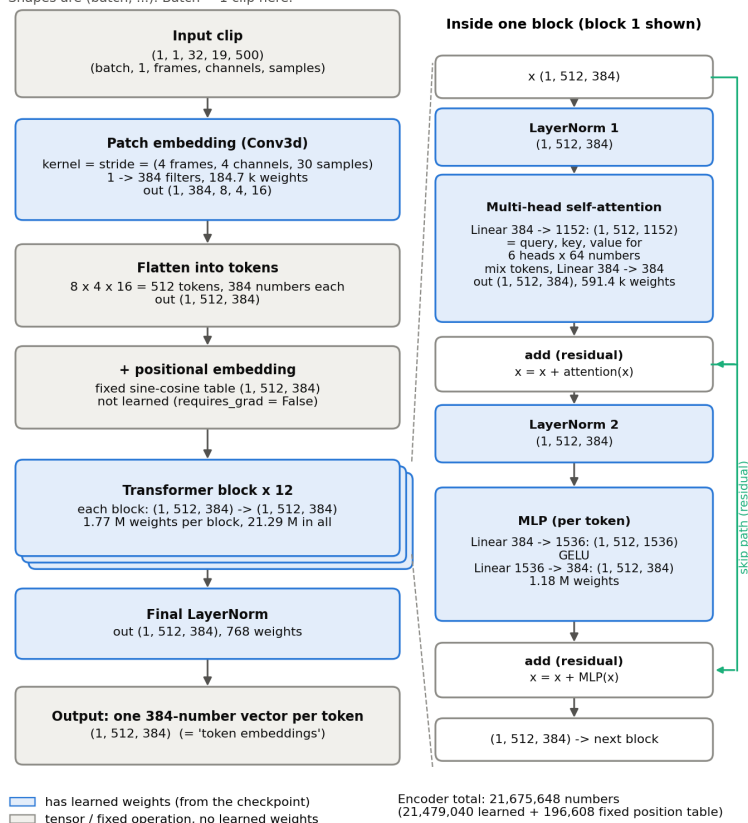
code/tuab_cached_eval.py, lines 74–78

```
def encoder(M, ckpt, seed):
    torch.manual_seed(seed)
    return init_model(DEV, ckpt, M["model_name"], patch_size=(4, 30), crop_size=224,
        frames_per_clip=M["frames_per_clip"], tubelet_size=M["tubelet_size"],
        use_sdpa=True, use_SiLU=False, tight_SiLU=False, uniform_power=True).eval()
```

The 512 tokens then pass through **12 transformer blocks** (ViT-M). Each block has **self-attention** with 6 heads, where every token can take information from every other token, followed by a small MLP (384 → 1,536 → 384). The output is again 512 tokens × 384: this is what the heads in "Downstream Evaluation" read. (ViT-B is larger: 768 numbers per token, 12 heads, 16-frame clips.)

Released ViT-M (4x30x4) encoder: shapes recorded on one real TUAB clip

Shapes are (batch, ...). Batch = 1 clip here.



Masking & The JEPA Paradigm

During pre-training, V-JEPA learns by predicting missing parts of the clip. **Masking** hides large blocks of tokens from the encoder.

The authors' mask generator (`src/masks/multiblock3d.py`). A block is a box in (time, channel group, sample group) set to 0. Several boxes are multiplied together; the zeros become the positions to predict (`mask_p`) and the ones the visible context (`mask_e`):

code/`src/masks/multiblock3d.py`, lines 149–164

```
def _sample_block_mask(self, b_size):
    t, h, w = b_size
    top = torch.randint(0, self.height - h + 1, (1,))
    left = torch.randint(0, self.width - w + 1, (1,))
    start = torch.randint(0, self.duration - t + 1, (1,))

    mask = torch.ones((self.duration, self.height, self.width), dtype=torch.int32)
    mask[start:start+t, top:top+h, left:left+w] = 0

    # Context mask will only span the first X frames
    # (X=self.max_context_frames)
    if self.max_context_duration < self.duration:
        mask[self.max_context_duration:, :, :] = 0

    # --
    return mask
```

code/`src/masks/multiblock3d.py`, lines 190–196

```
mask_e = torch.ones((self.duration, self.height, self.width), dtype=torch.int32)
for _ in range(self.npred):
    mask_e *= self._sample_block_mask(p_size)
mask_e = mask_e.flatten()

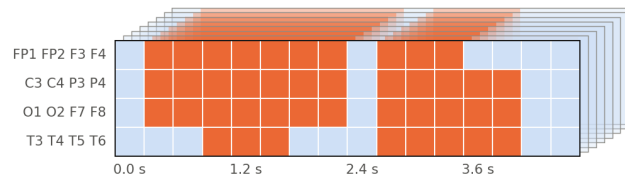
mask_p = torch.argwhere(mask_e == 0).squeeze()
mask_e = torch.nonzero(mask_e).squeeze()
```

Real masks from the authors' mask generator (ViT-M, $8 \times 4 \times 16 = 512$ tokens)

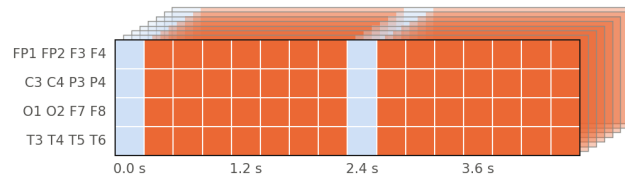
Blue = context: the encoder sees these tokens.

Orange = hidden: the predictor must guess the target encoder's vectors for these tokens.

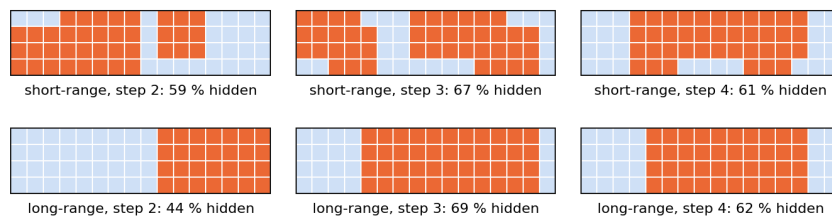
- (a) Short-range mask: 8 blocks, each asked to cover 15 % of the 4×16 slice
 -> each block is 3-4 rows x 2-3 columns and spans all 8 tubelet times
 (the stack behind the front slice).
 This step: context = 176 tokens, hidden = 336 of 512 (66 %).



- (b) Long-range mask: 2 blocks, each asked to cover 70 % of the 4×16 slice
 -> each block is all 4 rows x 5-8 columns and spans all 8 tubelet times
 (the stack behind the front slice).
 This step: context = 64 tokens, hidden = 448 of 512 (88 %).



- (c) Three more training steps: a new random mask every step (front slice only).



Average over 500 steps: 68 % hidden (short-range), 60 % (long-range). Blocks can overlap, so the share varies.

The Actual JEPA Block Diagram

The Joint Embedding Predictive Architecture (JEPA) fundamentally relies on an interaction between an **Encoder** and a **Predictor**:

1. **Context encoder**: sees only the **visible** tokens and encodes them.
2. **Target encoder (EMA)**: sees the **full, unmasked** clip. Its outputs at the **masked positions** are the targets. It is a slowly updated (EMA) copy of the context encoder.
3. **Predictor**: a small transformer that takes the context tokens plus "mask tokens" marking where the hidden positions are, and predicts the target encoder's outputs there.
4. **Loss**: the difference between predicted and target *embeddings*, never raw EEG values.

The training step in the authors' `app/vjepa/train.py`. The target encoder runs without gradients on the full clip; the context encoder sees only `masks_enc`; the predictor fills in `masks_pred`. The loss is $\text{mean } |z - h|^{\text{loss_exp}}$ (1.0 in the repository's EEG pre-training config, i.e. an L1 distance). After the optimiser step the target takes one EMA step:

`code/app/vjepa/train.py`, lines 428–455, 492–496

```
def forward_target(c):
    """
    Returns list of tensors of shape [B, N, D], one for each
    mask-pred.
    """
    with torch.no_grad():
        h = target_encoder(c)
        h = F.layer_norm(h, (h.size(-1),)) # normalize over feature-dim [B, N, D]
        # -- create targets (masked regions of h)
        h = apply_masks(h, masks_pred, concat=False)
        return h

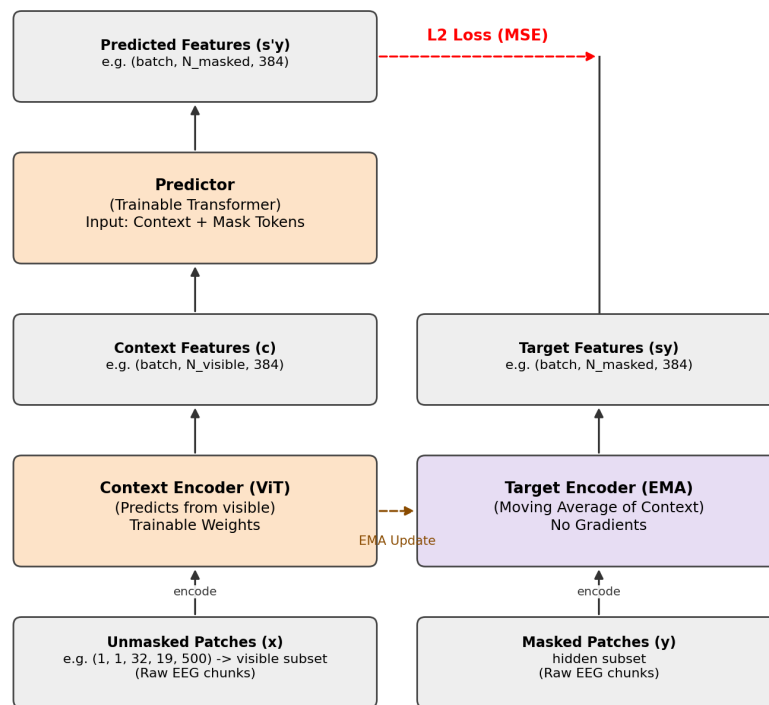
def forward_context(c, h):
    """
    Returns list of tensors of shape [B, N, D], one for each
    mask-pred.
    """
    z = encoder(c, masks_enc)
    z = predictor(z, h, masks_enc, masks_pred)
    return z

def loss_fn(z, h):
    loss = 0.
    # Compute loss and accumulate for each mask-enc/mask-pred pair
    for zi, hi in zip(z, h):
        loss += torch.mean(torch.abs(zi - hi)**loss_exp) / loss_exp
    loss /= len(masks_pred)
    return loss
    ...

# Step 3. momentum update of target encoder
m = next(momentum_scheduler)
with torch.no_grad():
    for param_q, param_k in zip(encoder.parameters(), target_encoder.parameters()):
        param_k.data.mul_(m).add_((1.-m) * param_q.detach().data)
```

V-JEPA: Joint Embedding Predictive Architecture

Shapes and blocks representing the pre-training process



The idea is that predicting embeddings, rather than the exact waveform, pushes the network to learn the important structure and ignore noise.

Collapse check. A known failure of JEPA training is **collapse**: the encoder gives nearly the same output for every input, which makes the task trivially easy. The released ViT-M shows this: its token standard deviation is very low and different recordings give almost identical embeddings (pairwise cosine near 1). This is why its frozen results are weak.

The check (`tuab_collapse_check.py`, input `T` = tokens of shape (recordings, 512, 384)): token spread within a recording, spread of the pooled embedding across recordings, and the mean pairwise cosine:

`code/tuab_collapse_check.py`, lines 44–49

```

def report(name, T, y):
    pooled = T.mean(1)
    tok_std = T.std(1).mean()
    rec_std = pooled.std(0).mean()
    z = pooled / np.linalg.norm(pooled, axis=1, keepdims=True)
    cos = (z @ z.T)[np.triu_indices(len(z), 1)].mean()
    # within-recording token spread
    # across-recording spread of pooled emb
  
```

FEI+C Architecture

FEI+C is our own two-branch model. It is made of two small encoders, each pre-trained **separately** on NMT with its own self-supervised task:

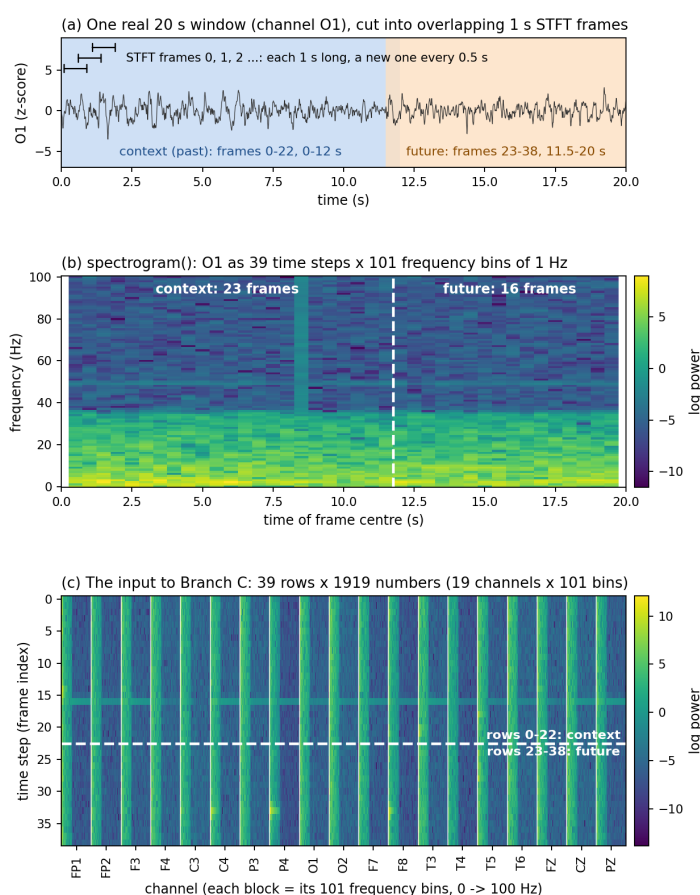
- **FEI** (Frequency-masked Embedding Inference, "Branch A") reads the raw waveform.
- **Branch C** reads a spectrogram and learns how the spectrum changes over time.

The two branches only meet at the very end, where their outputs are joined for the classifier (see "From encoders to a prediction" below). Act-0 always uses **plain FEI** (never Ada-FEI).

An **encoder** is a network that turns a piece of EEG into a short list of numbers, called an **embedding**. Two similar EEG pieces should give similar embeddings.

Branch C: the spectrogram branch

While the main FEI branch learns from raw EEG waveforms, **Branch-C** processes the **spectrogram**, which explicitly captures the frequency content (like alpha and beta bands) over time.



A **spectrogram** shows how much power each frequency has, moment by moment. Branch C builds it from a 20 s window (4,000 samples at 200 Hz):

- **Short-time FFT**: a 1 s window (nfft = 200) moves in 0.5 s steps (hop = 100). This gives **39 time steps**.
- **Frequencies**: each step has **101 frequency bins** of 1 Hz each (0–100 Hz).

- **Log-power:** the power is converted to log-power.
- **All channels:** the 19 channels are stacked, so each time step is described by $19 \times 101 = 1,919$ numbers.

The Branch-C settings used in Act-0 (h20_args: L = 4000, nfft = 200, hop = 100; the context is 60 % of the 39 steps = 23) and the spectrogram function:

code/head_to_head_cv.py, lines 37–38

```
def h20_args(): # Branch-C 20s config = the A+C=0.870 setting (--L 4000 --hop 100, rest default)
    return SimpleNamespace(d=256, h=128, L=4000, nfft=200, hop=100, tc_frac=0.6, enc="gru", tag="h20")
```

code/fei_branchC.py, lines 55–62

```
def spectrogram(x):
    """(B,19,L) -> (B, NT, 19*NF) log-power frames: a temporal SEQUENCE of spectra."""
    B_, C, _ = x.shape
    z = torch.stft(x.reshape(B_ * C, L), n_fft=NFFT, hop_length=HOP, win_length=NFFT,
                    window=WIN.to(x.device), center=False, return_complex=True) # (B*C, NF, T)
    p = (z.real ** 2 + z.imag ** 2 + 1e-6).log() # log-power
    p = p.reshape(B_, C, NF, -1).permute(0, 3, 1, 2) # (B, T, C, NF)
    return p.reshape(B_, p.shape[1], C * NF) # (B, T, C*NF)
```

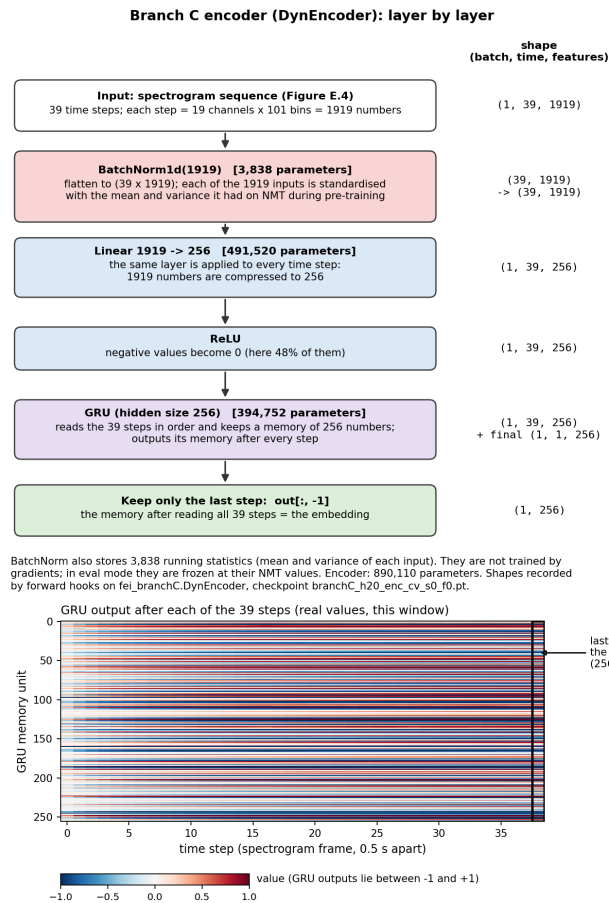
Branch C's encoder (DynEncoder) then processes the 39 steps in order:

1. **BatchNorm:** each of the 1,919 inputs is shifted and scaled, using a mean and variance remembered from NMT. This keeps every input near 0 with spread about 1. If new data has a very different spectrum, these remembered values no longer fit, and the inputs become huge. This is exactly what happened on the stroke data (Section "Stroke", clean re-prep).
2. **Linear 1,919 → 256, then ReLU:** this compresses each step into 256 numbers.
3. **GRU (256):** a recurrent layer reads the 39 steps one after another and keeps a running "memory". Its final memory state (256 numbers) is the window's embedding.

code/fei_branchC.py, lines 65–79

```
class DynEncoder(nn.Module):
    """Per-frame spectral embed + GRU over time -> summary of a frame span. (B,T,IN_DIM)->(B,d)."""
    def __init__(self, d=256, in_dim=None):
        super().__init__()
        in_dim = in_dim or IN_DIM # read the (possibly reconfigured) global at build time
        self.bn = nn.BatchNorm1d(in_dim)
        self.embed = nn.Linear(in_dim, d)
        self.gru = nn.GRU(d, d, batch_first=True)

    def forward(self, z): # z: (B,T,in_dim)
        B_, T, Din = z.shape
        h = self.bn(z.reshape(B_ * T, Din)).reshape(B_, T, Din)
        h = torch.relu(self.embed(h))
        out, _ = self.gru(h) # (B,T,d)
        return out[:, -1] # last hidden = summary of the span
```



Branch-C JEPA (DynJEPA)

Branch C is trained on its own, independently of FEI. Its self-supervised task is "predict the future from the past":

- **Context encoder:** reads the first **23 time steps** (the past) and produces an embedding.
- **Target encoder:** a slowly updated copy of the encoder, called the EMA copy. It reads the last **16 steps** (the future) and produces the target embedding.
- **Predictor:** a small network that tries to turn the past embedding into the future one.

The loss compares *embeddings* (L2-normalised MSE), never raw spectrogram values. **EMA** (exponential moving average) means that after every training step the target's weights move 0.4 % of the way towards the context encoder's (factor 0.996). The target therefore changes slowly and gives stable targets.

DynJEPA. forward splits the steps at TC; the target runs under `no_grad`, and `ema` is the slow update:

code/fei_branchC.py, lines 142–152

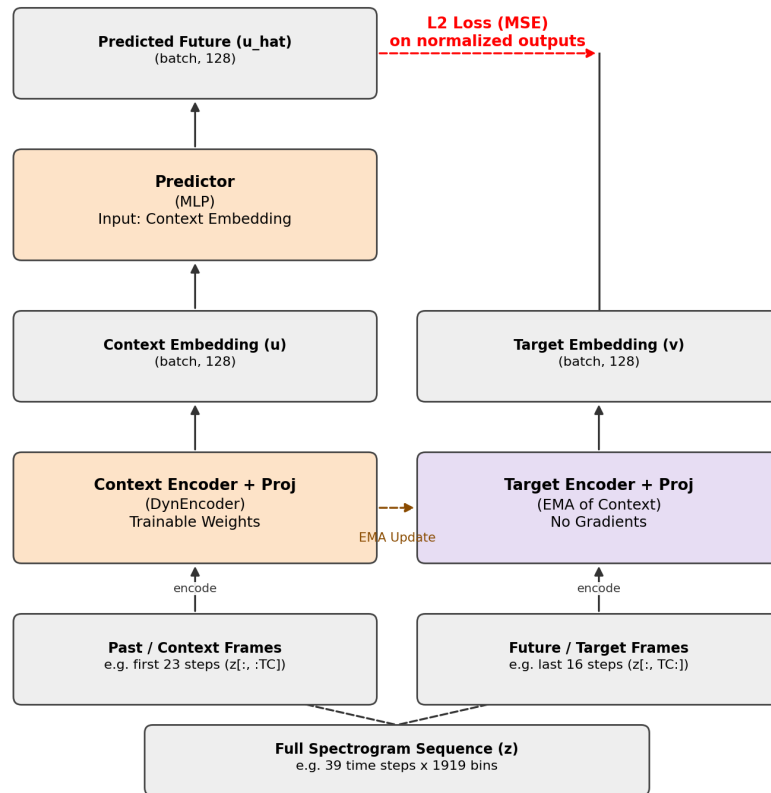
```
@torch.no_grad()
def ema(self, a=0.996):
    for s, t in zip(self._src(), self._tgt()): t.mul_(a).add_(s, alpha=1 - a)

def forward(self, z):
    past, future = z[:, :TC], z[:, TC:]
    u = self.proj(self.enc(past))
    with torch.no_grad():
        v = self.proj_t(self.enc_t(future))
    nrm = lambda a: NF_.normalize(a, dim=-1)
    return NF_.mse_loss(nrm(self.pred(u)), nrm(v))
```

Self-supervised means no labels are used. The data itself provides the task.

Branch-C JEPA: Spectral-Dynamics (DynJEPA)

Predicts the future-span embedding from the past-span context



FEI (Branch A): encoder and frequency masking

FEI's encoder is a small **1-D convolutional network**, not a transformer. Its input is a 20 s window of 19 channels $\times$ 4,000 samples (200 Hz). It has four blocks, each one Conv1d (kernel 7, stride 2) $\rightarrow$ BatchNorm $\rightarrow$ GELU:

- The number of channels grows 19 $\rightarrow$ 64 $\rightarrow$ 128 $\rightarrow$ 128 $\rightarrow$ 256.
- Stride 2 halves the time length in every block.
- An average over time then gives 256 numbers, and a final Linear layer outputs the **256-number embedding**.

A convolution slides a small learned filter along time, so each block detects local waveform patterns.

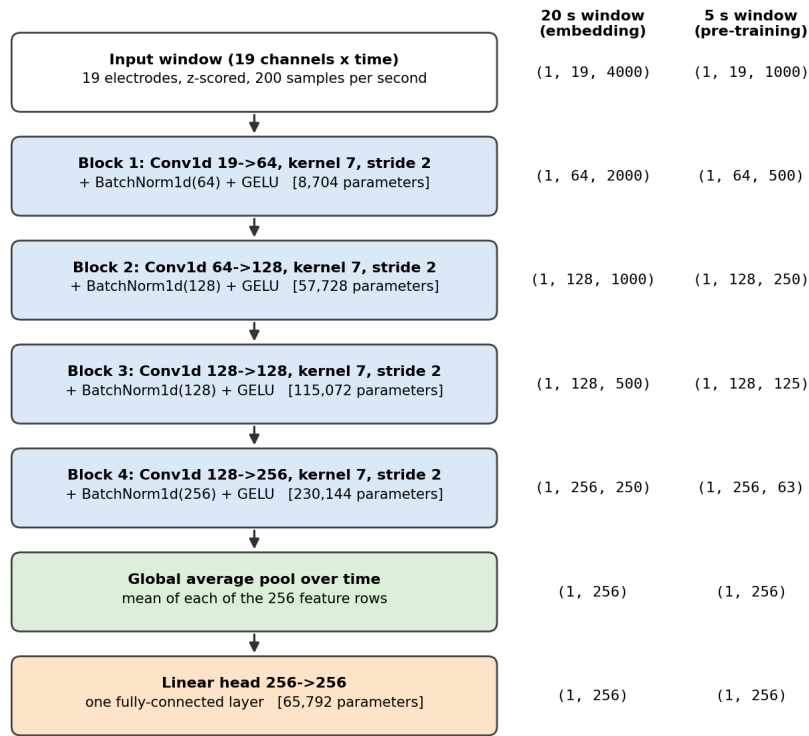
code/fei_pretrain.py, lines 88–101

```

class Encoder(nn.Module):
    """(B,19,L) -> (B,d). Small strided 1D conv stack + global average pool."""
    def __init__(self, d=256, ch=NCH):
        super().__init__()
        w = [ch, 64, 128, 128, 256]
        blocks = []
        for a, b in zip(w[:-1], w[1:]):
            blocks += [nn.Conv1d(a, b, 7, stride=2, padding=3), nn.BatchNorm1d(b), nn.GELU()]
        self.conv = nn.Sequential(*blocks)
        self.head = nn.Linear(w[-1], d)

    def forward(self, x):
        h = self.conv(x).mean(-1)  # global avg pool over time
        return self.head(h)
  
```

FEI encoder (Branch A): layer by layer



Output: one embedding of 256 numbers per window, whatever the window length.
Shapes are (batch, channels or features, time steps), recorded by forward hooks on fei_pretrain.Encoder with checkpoint fei_enc_cv_s0_f0.pt. Each block halves the time axis; one step after block 4 sees 91 input samples = 0.46 s. Encoder: 477,440 parameters.

FEI's masking hides **frequencies, not time**. For each training window:

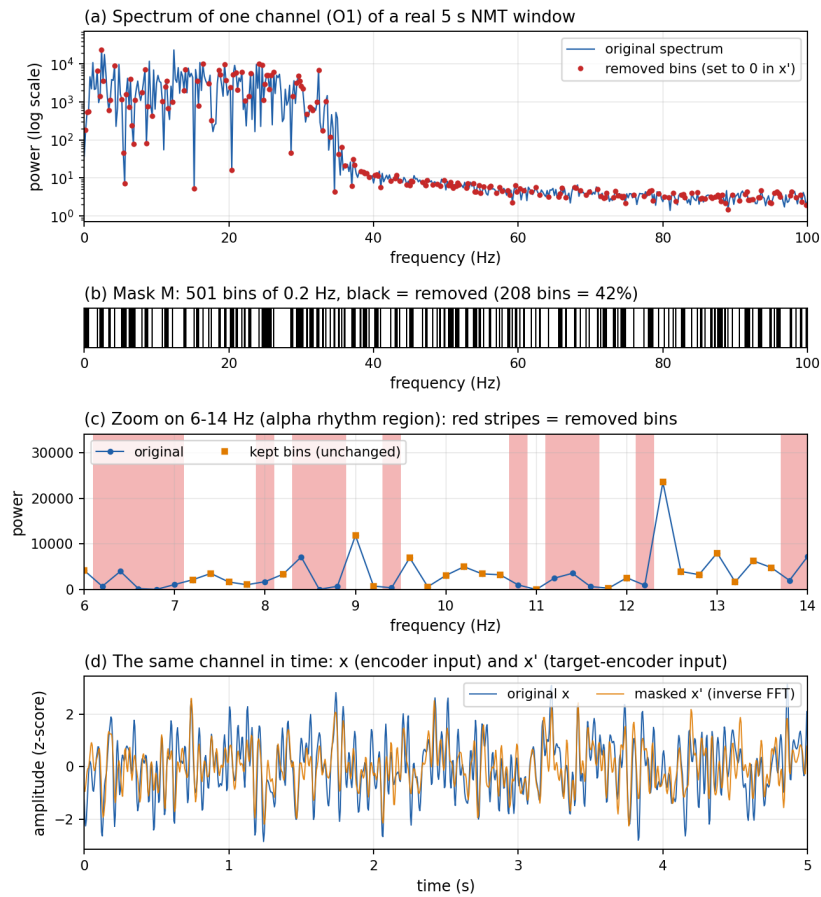
1. Take the FFT of every channel. The FFT turns the signal into its list of frequency components.
2. Pick a random fraction between 0 % and 70 % of the frequency bins. Set them to zero, using the same bins on all 19 channels.
3. Transform back to a waveform (inverse FFT). The result looks like the same EEG with some rhythms removed.

freq_mask. Act-0 uses plain FEI, which is bias = 0: the else branch, p = ratio, so every bin has the same chance of being masked. The if bias branch is Ada-FEI and is not used in Act-0:

code/fei_pretrain.py, lines 155–175

```
def freq_mask(x, d=None, bias=0.0, b1=0.0, b2=0.7):
    """Zero a random U(b1,b2) fraction of rFFT bins (shared across channels).
    Ada-FEI: when bias>0, re-weight per-bin masking toward high-difficulty bins
    using the EMA difficulty map d (n,) -- the adaptive/curriculum frequency
    masking that plain FEI (bias=0, uniform) lacks. bias=0 -> IDENTICAL to plain
    FEI (clean ablation). E[fraction masked] stays ~ratio regardless of bias, so
    the knob changes WHERE bands are masked, not how many.
    Returns masked signal x' (B,19,L) and mask M (B,n) in {0,1}."""
    B, _, L = x.shape
    Xf = torch.fft.rfft(x, dim=-1) # (B,19,n) complex
    n = Xf.shape[-1]
    ratio = torch.empty(B, 1, device=x.device).uniform_(b1, b2)
    if bias and d is not None and float(d.std()) > 0:
        z = ((d - d.mean()) / (d.std() + 1e-6)).clamp(-5, 5) # standardized difficulty, capped
        w = torch.exp(bias * z) # emphasize hard/informative bins
        p = (ratio * w / w.mean()).clamp(0, 1) # (B,n), keeps E[frac]~ratio
    else:
        p = ratio # uniform == plain FEI
    M = (torch.rand(B, n, device=x.device) < p).float() # 1 = masked
    xp = torch.fft.irfft(Xf * (1 - M).unsqueeze(1), n=L, dim=-1)
    return xp.float(), M
```

The figure shows this on a real NMT window.



FEI pre-training (the JEPA task)

FEI is pre-trained with a **JEPA** task (Joint-Embedding Predictive Architecture: predict an *embedding*, not the raw signal). **Branch C plays no part in it.** One training step works like this:

1. **Online encoder:** reads the **original** window and gives embedding u .
2. **Target encoder** (the EMA copy): reads the **frequency-masked** window and gives the target u' .
3. **Mask prompt:** a small layer turns "which bins were removed" into a vector m .
4. **Predictor 1:** from $u + m$, predict u' . The question it answers: "what will the EEG look like to the target encoder once these frequencies are removed?"
5. **Predictor 2:** from $u - u'$, predict m . The question: "which frequencies were removed?"
6. **Loss:** the sum of both errors, using L2-normalised MSE. Then the target encoder takes one EMA step (0.996).

To succeed, the embedding has to "know" the spectral content of the EEG.

FEI. forward is steps 1–6 (x = original window, xp = masked window, M = mask), and ema is the target update:

code/fei_pretrain.py, lines 135–152

```
@torch.no_grad()
def ema(self, a=0.996):
    for s, t in zip(self._src_params(), self._tgt_params()):
        t.mul_(a).add_(s, alpha=1 - a)

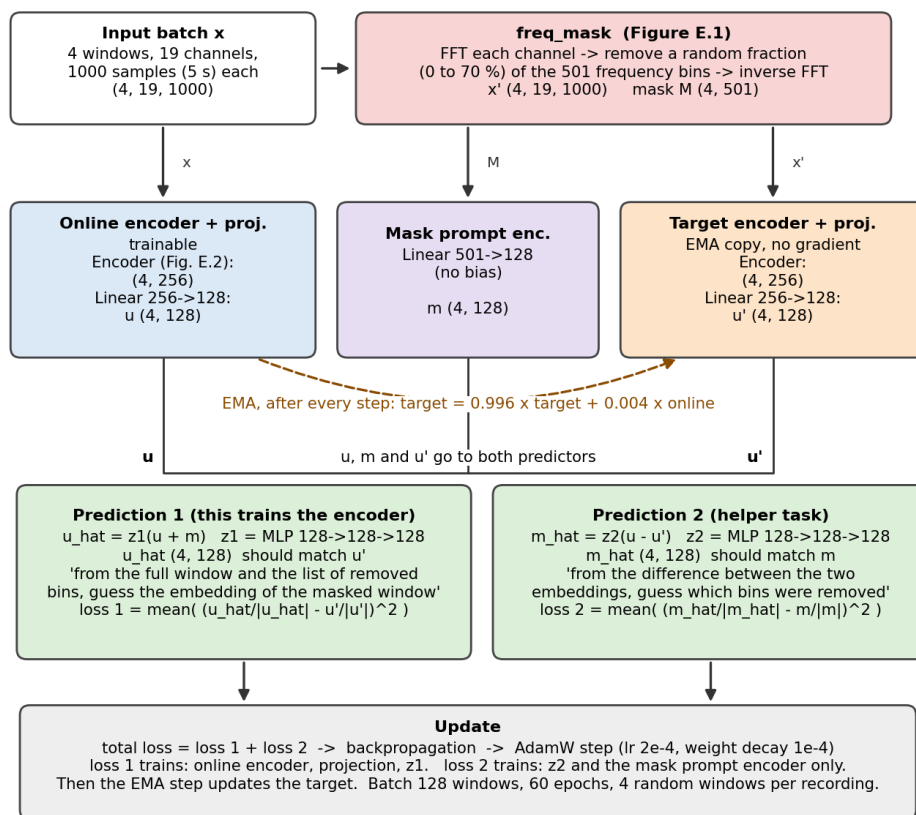
def forward(self, x, xp, M):
    e = self.enc(x); u = self.proj(e)           # online, original window
    with torch.no_grad():
        up = self.proj_t(self.enc_t(xp))       # momentum, masked window
    m = self.mask_enc(M)
    uhat = self.z1(u + m.detach())              # predict masked embedding
    mhat = self.z2(u.detach() - up)            # predict which bands removed
    # L2-normalize pred & target before MSE (BYOL-style) -> bounded loss, no
    # scale collapse. Equivalent to 2 - 2*cos(pred, target).
    nrm = Lambda a: F.normalize(a, dim=-1)
    l_embed = F.mse_loss(nrm(uhat), nrm(up), reduction="none").mean(-1) # (B,) per-sample
    loss = l_embed.mean() + F.mse_loss(nrm(mhat), nrm(m))
    return loss, e, l_embed.detach()           # psl feeds the Ada difficulty map
```

One training step inside pretrain (for plain FEI args.mask_bias is 0):

code/fei_pretrain.py, lines 238–241

```
x = x.to(DEV, non_blocking=True)
xp, M = freq_mask(x, d, args.mask_bias)
loss, _, psl = model(x, xp, M)
opt.zero_grad(); loss.backward(); opt.step(); model.ema(args.ema)
```

One FEI pre-training step (real shapes, batch of 4 windows)



From encoders to a prediction (FEI+C readout)

After pre-training, both encoders are **frozen**: their weights are never changed again. A whole recording becomes one prediction like this:

1. **Windows**: cut the recording into non-overlapping 20 s windows.
2. **Average**: run each window through FEI (256 numbers) and through Branch C (256 numbers), then average over all windows of the recording.
3. **Join**: concatenate the two vectors into **512 numbers** per recording. This is the only place where the two branches meet (late fusion).
4. **Classify**: apply StandardScaler (each feature is shifted to mean 0 and scaled to spread 1), then **logistic regression** (C = 1, balanced class weights). This gives a probability of "abnormal" (TUAB) or "stroke" (Phase B).
5. **Ensemble**: we have 5 pre-trained versions of each encoder, one per NMT cross-validation fold (f0–f4). Their 5 probabilities are averaged.

In `tuab_feic.py`, `feat` builds each arm's features (FEI+C = the two 256-number blocks side by side). The probe fits one classifier per fold encoder and averages the 5 probabilities (`P_.mean(0)`):

code/tuab_feic.py, lines 111, 120–123

```
feat = lambda arm, k: {"FEI": E[f"fei_{k}"], "C": E[f"c_{k}"], "FEI+C": np.hstack([E[f"fei_{k}"],  
E[f"c_{k}"]])}[arm]  
...  
for arm in ("FEI", "C", "FEI+C"):  
    for kind, ks in (("", ENCODERS[:5]), ("rand-", ENCODERS[5:])):  
        P_ = np.array([proba(feat(arm, k)[tr], y[tr], feat(arm, k)[ev]) for k in ks])  
        p = P_.mean(0); au = [metrics(y[ev], q)["auroc"] for q in P_]
```

Random-init control: the same pipeline is also run with *untrained* (randomly initialised) encoders. If the pre-trained encoders do not beat these, pre-training did not help.

How `embed` in `tuab_feic.py` builds the encoders: fold encoders f0–f4 load their NMT weights, while `rand0`–`rand2` keep the random weights drawn after `torch.manual_seed`:

code/tuab_feic.py, lines 87–92

```
torch.manual_seed(int(k[-1]))  
a, c = F.Encoder(args.d).to(F.DEV), C._enc(args)(args.d).to(F.DEV)  
if k.startswith("f"):  
    a.load_state_dict(torch.load(f"{ROOT}/code/fei{TAG}_enc_cv_s0_{k}.pt", map_location=F.DEV))  
    c.load_state_dict(torch.load(f"{ROOT}/code/branchC_h20{TAG}_enc_cv_s0_{k}.pt", map_location=F.DEV))  
a.eval(); c.eval()
```

Downstream Evaluation Architectures

A pre-trained encoder only produces embeddings. To get a decision (normal vs abnormal), a small classifier called a **head** is put on top. We use three ways of training it, and each answers a different question:

Setting	What is trained	Question it answers
Frozen + head (a "probe")	only the head; encoder weights fixed	Are the pre-trained features themselves useful? (the paper's main TUAB result)
Fine-tune	encoder and head, end to end	How good can the pre-trained network become on this task?
From scratch	same network and head, starting from random weights	What do we get with no pre-training? (fine-tune minus scratch = the value of pre-training)

Results are reported as **AUROC**, the area under the ROC curve. It is the probability that a random abnormal recording gets a higher "abnormal" score than a random normal one: 0.5 is chance and 1.0 is perfect.

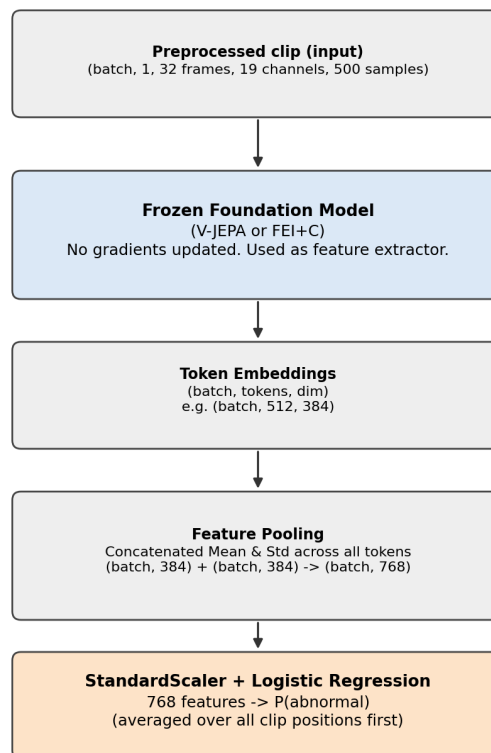
What the released EEG-VJEPA checkpoints are. The paper's authors pre-trained them; in Act-0 we did **not** pre-train EEG-VJEPA ourselves. We also found that the released ViT-M gives almost the same tokens for every recording. This is called "collapse", and it is measured by a very low token standard deviation and a pairwise cosine similarity near 1. So any gain from its features is expected to be small.

1. Linear Probe (Logistic Regression)

The **Linear Probe** is the simplest head network. It is used to strictly evaluate the quality of the frozen features produced by the foundation models.

Downstream Linear Probe Architecture

Used for frozen evaluation on TUAB



- **How it works (ViT-M):**

- The encoder is frozen.
- For every clip position, we take the **mean and standard deviation over the 512 tokens** (384 + 384 = 768 numbers).
- We then average these over all clip positions of the recording, which gives one 768-number vector per recording.

- **How it works (FEI+C):** the features are the 512 numbers from the FEI+C readout (FEI 256 + C 256), not tokens.

- **The head:** StandardScaler followed by LogisticRegression (C = 1, class_weight="balanced"), trained with scikit-learn.

- **Where it is used:**

- tuab_vitm_linear.py and tuab_feic.py on TUAB;
- stroke_phaseB.py on the stroke data.

- **Why:** the same simple head for every encoder gives a fair comparison, because a powerful head cannot hide weak features.

The ViT-M pooling (tuab_vitm_linear.py) and the head (tuab_feic.py; tuab_vitm_linear.py and stroke_phaseB.py build the same scaler + logistic-regression pipeline):

code/tuab_vitm_linear.py, lines 32–33

```
def pool(tok): # (P, tokens, D) -> (2D,)
    return np.concatenate([tok.mean(1), tok.std(1)], -1).mean(0)
```

code/tuab_feic.py, lines 97–99

```
def proba(Xtr, ytr, Xte):
    clf = make_pipeline(StandardScaler(), LogisticRegression(C=1.0, class_weight="balanced",
max_iter=5000))
    return clf.fit(Xtr, ytr).predict_proba(Xte)[: , 1]
```

2. Attention Classifier (the paper's §4.2 head: frozen probe and fine-tuning)

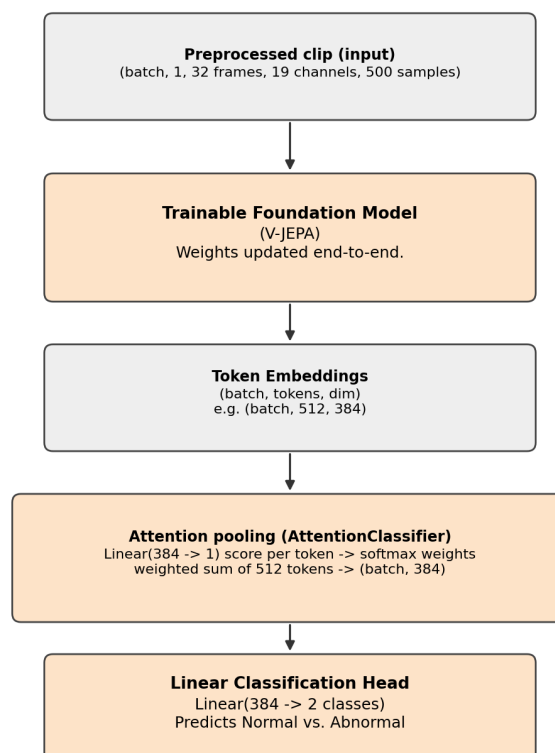
This is the head from the authors' own evaluation code (`AttentionClassifier`, `src/models/attentive_pooler.py`). We use it in two places:

- **Frozen:** on the frozen released encoder (the main Part-1 table, `tuab_cached_eval.py`);
- **Fine-tune and scratch:** as the head of the fine-tuned and from-scratch networks (`tuab_finetune_fast.py`).

The authors' recipe is 500 epochs, batch size 2 and learning rate $1e-3$.

Downstream Attention Classifier (Fine-Tuning)

Used for end-to-end fine-tuning on TUAB



- **The head is very small.** It has only two layers:
- **Score:** a Linear(384 → 1) layer gives each of the 512 tokens one score.
- **Weights:** softmax turns the 512 scores into weights that add up to 1.
- **Weighted average:** the tokens are averaged with these weights, giving 384 numbers. The head thus learns *which tokens to listen to*.
- **Classify:** a Linear(384 → 2) layer gives the normal / abnormal scores.
- **Fine-tune:** all ViT-M weights are updated together with the head, starting from the released checkpoint.
- **From scratch:** exactly the same, but the ViT-M starts from random weights.

The authors' code:

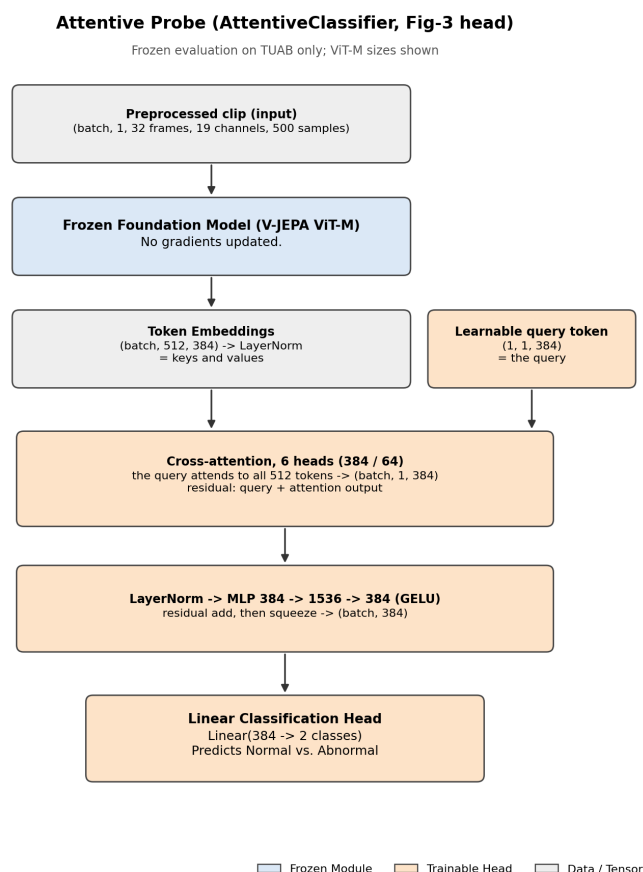
code/src/models/attentive_pooler.py, lines 156–169

```
class AttentionClassifier(nn.Module):
    """ Attention Classifier """
    def __init__(self, embed_dim=768, num_classes=1000):
        super().__init__()
        self.linear = nn.Linear(embed_dim, num_classes, bias=True)
        self.attention = nn.Linear(embed_dim, 1)

    def forward(self, x):
        # Compute attention scores and apply softmax
        weights = torch.softmax(self.attention(x), dim=1)
        # Weighted sum of embeddings
        x = torch.sum(weights * x, dim=1)
        x = self.linear(x)
        return x
```

3. Attentive probe (the paper's Fig-3 head, frozen only)

The second frozen head (AttentionClassifier) is larger. It is a cross-attention block (one layer, 6 attention heads for ViT-M: 384 / 64), where a learnable **query token** "asks" all 512 tokens for information, followed by a linear layer. It is only used on the frozen encoders, as the second column of the Part-1 table.



The difference from the §4.2 head: there, each token gets a score from a single Linear layer. Here, the query token is compared with every token through full multi-head attention (queries, keys and values), and the result then passes through an MLP. This gives the head many more trainable weights.

The authors' code (AttentivePooler, in the same file, holds the query token and the cross-attention block):

code/src/models/attentive_pooler.py, lines 105–136

```
class AttentiveClassifier(nn.Module):
    """ Attentive Classifier """
    def __init__(
        self,
        embed_dim=768,
        num_heads=12,
        mlp_ratio=4.0,
        depth=1,
        norm_layer=nn.LayerNorm,
        init_std=0.02,
        qkv_bias=True,
        num_classes=1000,
        complete_block=True,
    ):
        super().__init__()
        self.pooler = AttentivePooler(
            num_queries=1,
            embed_dim=embed_dim,
            num_heads=num_heads,
            mlp_ratio=mlp_ratio,
            depth=depth,
            norm_layer=norm_layer,
            init_std=init_std,
            qkv_bias=qkv_bias,
            complete_block=complete_block,
        )
        self.linear = nn.Linear(embed_dim, num_classes, bias=True)

    def forward(self, x):
        x = self.pooler(x).squeeze(1)
        x = self.linear(x)
        return x
```

Summary of Integration

- **Linear probe:** [Preprocessed clips] -> [Frozen encoder] -> [Mean/Std pooling, averaged per recording] -> [StandardScaler + Logistic Regression] -> [Prediction]
- **Frozen §4.2 / Fig-3 probe:** [Preprocessed clip] -> [Frozen ViT-M] -> [512 tokens x 384] -> [AttentionClassifier or AttentiveClassifier] -> [Prediction]
- **Fine-tune / scratch:** [Preprocessed clip] -> [Trainable ViT-M] -> [512 tokens x 384] -> [AttentionClassifier] -> [Prediction]